

MÉMOIRE DE RECHERCHE

Montée en charge de la plateforme de correction automatique CodEval

*Étude de quatre architectures de traitement et proposition d'une architecture fondée
sur une file de tâches persistante en base de données*

Présenté par

DAGBO KADY CHRIST-PHANUEL

20 septembre 2026

Sommaire

Introduction générale

Chapitre 1 : Contexte et problématique

- 1.1 Évaluation automatique et CodEval
- 1.2 Problématique de la charge de travail
- 1.3 Question de recherche et hypothèses

Chapitre 2 : Systèmes étudiés

- 2.1 Traitement séquentiel (architecture actuelle)
- 2.2 Parallélisme sur une machine : pool de processus local (architecture A)
- 2.3 Workers distribués autour d'un courtier de messages (architecture B)
- 2.4 Messagerie serverless élastique (architecture C)
- 2.5 Discussion sur les limites

Chapitre 3 : Architecture proposée

- 3.1 Bilan des architectures étudiées
- 3.2 Des limites aux exigences de conception
- 3.3 Principe de l'architecture proposée
- 3.4 Description de l'architecture
- 3.5 Comparaison avec les architectures étudiées

Chapitre 4 : Méthodologie expérimentale

- 4.1 Matériel et logiciels
- 4.2 Données corrigées pendant les mesures
- 4.3 Construction du jeu de données
- 4.4 Ce que nous mesurons
- 4.5 Mesure de l'architecture actuelle (référence)
- 4.6 Mesure de l'architecture A (pool de processus local)
- 4.7 Mesure de l'architecture B (workers distribués via Redis)
- 4.8 Mesure de l'architecture proposée (P)
- 4.9 Pourquoi l'architecture C n'est pas mesurée
- 4.10 Limites du dispositif

Chapitre 5 : Résultats

- 5.1 État d'avancement
- 5.2 Vérification du dispositif
- 5.3 Mesures déjà disponibles : effet de la granularité
- 5.4 à 5.9 Résultats des expériences X1 à X6

- 5.10 Décision sur les critères
- 5.11 Décision sur les hypothèses

Chapitre 6 : Discussion

- 6.1 Analyse des résultats
- 6.2 Vérification des hypothèses
- 6.3 Limites et implications

Conclusion générale

Références bibliographiques

Annexes A à E

Table des matières

Liste des tableaux

Les tableaux sont numérotés par chapitre. Ceux des annexes portent la lettre de leur annexe.

Tableau	Intitulé
Tableau 3.1	Bilan des architectures étudiées
Tableau 3.2	Comparaison de l'architecture proposée avec les architectures étudiées
Tableau 4.1	Poste de mesure
Tableau 4.2	Logiciels utilisés et leur rôle
Tableau 4.3	Exercices de l'évaluation de référence BENCH-SRIT
Tableau 4.4	Classes de réponses et leur part dans les copies
Tableau 4.5	Profils de charge
Tableau 4.6	Fichiers produits par le tirage des copies
Tableau 4.7	Lignes créées en base par le chargement du jeu de données
Tableau 4.8	Question posée par chaque expérience
Tableau 4.9	Variables de l'expérimentation
Tableau 4.10	Indicateurs mesurés
Tableau 4.11	Seuils de réussite
Tableau 4.12	Plan d'ensemble des mesures
Tableau 4.13	Ce qui change d'une architecture à l'autre dans le déroulement d'une mesure
Tableau 5.1	État d'avancement des expériences
Tableau 5.2	Éléments du dispositif vérifiés
Tableau 5.3	Débit selon la granularité et le nombre de correcteurs
Tableau 5.4	Temps total par charge (X1)
Tableau 5.5	Gain par worker ajouté (X2)
Tableau 5.6	Passage à deux nœuds (X3)
Tableau 5.7	Reprise après panne (X4)
Tableau 5.8	Exactitude des notes (X5)
Tableau 5.9	Réactivité de l'API (X6)
Tableau 5.10	Décision sur les critères R1 à R6
Tableau 5.11	Décision sur les hypothèses
Tableau B.1	Contenu de l'évaluation BENCH-SRIT
Tableau C.1	Scripts de construction du jeu de données
Tableau C.2	Réglages du tirage des copies
Tableau E.1	Scripts du banc de mesure
Tableau E.2	Options de l'orchestrateur

Liste des figures

Les figures sont numérotées par chapitre.

Figure	Intitulé
Figure 1.1	Architecture générale de la plateforme CodEval
Figure 2.1	Traitement séquentiel : organisation et déroulement
Figure 2.2	Pool de processus local sur une machine multiprocesseur (architecture A)
Figure 2.3	Dispositif à courtier de messages mesuré par Ibrahim et al. (architecture B)
Figure 2.4	Messagerie serverless élastique et workers éphémères (architecture C)
Figure 3.1	Traduction des limites des architectures étudiées en exigences de conception
Figure 3.2	Vue d'ensemble de l'architecture proposée (P)
Figure 3.3	Effet de la granularité sur l'équilibre de charge
Figure 3.4	États d'une tâche de correction
Figure 3.5	Séquence de traitement d'une campagne
Figure 3.6	Les deux modes de déploiement de l'architecture proposée
Figure 4.1	Dispositif expérimental sur le poste de mesure
Figure 4.2	Les huit étapes d'une mesure

Glossaire

Terme	Définition
Accélération	Rapport entre le temps de la correction séquentielle et celui d'une autre organisation, à charge égale. Notée $S(N)$.
Bail	Durée pendant laquelle un correcteur garde une tâche réservée. S'il ne la renouvelle pas, la tâche redevient disponible pour un autre.
Banc de mesure	Ensemble des scripts qui préparent, déclenchent et observent une correction sans y participer.
Bridage thermique	Baisse de fréquence que le système décide lui-même quand la puce chauffe, sans message d'erreur. Elle fausse une mesure.
Campagne de correction	Un passage de correction sur une évaluation donnée, de son déclenchement à la dernière note écrite.
Chauffe	Première répétition d'une configuration, lancée puis écartée de l'analyse, le temps que caches et fichiers atteignent leur état courant.
Charge	Nombre de copies à corriger dans une mesure.
Copie	Ensemble des réponses d'un étudiant à une évaluation. Une copie compte ici cinq réponses.
Correcteur	Processus qui corrige. Synonyme de worker dans ce document.
Courtier de messages	Composant intermédiaire qui reçoit des tâches et les distribue à des consommateurs. Redis, RabbitMQ et NATS en sont.
Débit	Nombre de copies corrigées par minute.
Échantillonneur	Outil qui relève à intervalle régulier le processeur, la mémoire et les attentes de verrous, sans intervenir dans la correction.
Efficacité	Accélération divisée par le nombre de workers. Une efficacité de 1 signifie que chaque worker ajouté apporte tout son travail.
Empreinte SHA-256	Suite de caractères calculée à partir d'un fichier. Deux fichiers identiques donnent la même empreinte, la moindre modification la change.
Goulot d'étranglement	Étape qui limite le débit de l'ensemble, indépendamment de la capacité des autres.
Granularité	Taille de l'unité de travail distribuée : une copie entière, ou une réponse à un exercice.
Idempotence	Propriété d'un traitement qui, relancé sur une unité déjà traitée, ne crée pas de doublon.
Latence	Pour une copie, temps écoulé entre le déclenchement de la campagne et l'écriture de sa dernière note.
Nœud	Une machine participant à la correction.
Orchestrateur	Script qui enchaîne les étapes d'une mesure : création de la base, démarrage, déclenchement, attente, export, refroidissement.
Percentile	Valeur en dessous de laquelle tombe un pourcentage donné des observations. Le p95 est dépassé par une observation sur vingt.
Rééchantillonnage	Méthode d'estimation d'un intervalle de confiance par tirages répétés avec remise dans les mesures obtenues. Dite bootstrap.
Réservation sans attente	Clause de PostgreSQL par laquelle un processus prend une ligne libre et ignore celles déjà prises, au lieu d'attendre.

Terme	Définition
Sandbox	Environnement d'exécution isolé où tourne le code de l'étudiant, avec des limites de temps et de mémoire.
Tâche	Unité de travail inscrite dans la file, réservée par un correcteur puis marquée terminée.
Temps mural	Temps écoulé à l'horloge, par opposition au temps processeur consommé.
Transpilation	Traduction d'un code source d'un langage vers un autre. Ici, du pseudo-code algorithmique vers Python, pour l'exécuter.
Verrou	Mécanisme par lequel la base empêche deux transactions de modifier la même ligne en même temps.
Worker	Processus séparé de l'API, qui prend les corrections en attente et les traite.

Sigles et acronymes

Sigle	Signification
API	Interface de programmation applicative (<i>Application Programming Interface</i>)
CPU	Processeur (<i>Central Processing Unit</i>)
CSV	Format de fichier tabulaire à valeurs séparées par des virgules (<i>Comma-Separated Values</i>)
DSN	Chaîne de connexion à une base de données (<i>Data Source Name</i>)
ESATIC	École Supérieure Africaine des Technologies de l'Information et de la Communication
IC	Intervalle de confiance
JSON	Format d'échange de données textuel (<i>JavaScript Object Notation</i>)
NoSQL	Famille de bases de données non relationnelles
QCM	Questionnaire à choix multiples
RSS	Mémoire résidente d'un processus (<i>Resident Set Size</i>)
SHA-256	Fonction d'empreinte cryptographique sur 256 bits (<i>Secure Hash Algorithm</i>)
SQL	Langage d'interrogation des bases relationnelles (<i>Structured Query Language</i>)
SRIT	Filière de l'ESATIC dont relève le cours d'algorithmique servant de référence aux exercices
SSD	Disque à mémoire flash (<i>Solid State Drive</i>)
USL	Loi de passage à l'échelle universelle (<i>Universal Scalability Law</i>)

Les notations suivantes reviennent dans tout le document.

Notation	Sens
Seq, A, B, C, P	Les cinq organisations comparées : séquentielle, pool de processus local, workers distribués via un courtier, messagerie infogérée, architecture proposée
H1, H2, H3, HP	Les hypothèses de la recherche, énoncées au chapitre 1
X1 à X7	Les expériences du plan de mesure, définies en section 4.4.1
M1 à M12	Les indicateurs mesurés, définis en section 4.4.2
R1 à R6	Les critères de réussite, fixés en section 4.4.3
K1 à K5	Les cinq classes de réponses d'étudiants, définies en section 4.2.2
N	Nombre de workers par nœud
T _{total}	Temps total d'une campagne de correction
S(N), E(N)	Accélération et efficacité obtenues avec N workers
p50, p95	Percentiles 50 et 95 des latences par copie

Introduction générale

Dans l'enseignement supérieur, les évaluations pratiques de programmation occupent une place importante, mais corriger à la main le code de chaque étudiant prend beaucoup de temps et varie d'un correcteur à l'autre. Pour résoudre ce problème, des plateformes de correction automatique ont vu le jour, dont CodEval, une application web qui permet aux enseignants de créer des évaluations et qui corrige les copies en exécutant le code des étudiants contre des tests préparés à l'avance. CodEval repose sur une API web construite avec FastAPI, qui gère les utilisateurs, les évaluations et les soumissions, et sur un worker de correction qui récupère les corrections en attente dans la base de données. Aujourd'hui, ce worker traite les copies l'une après l'autre, ce qui convient à de petits groupes mais pose la question de ce qui arrive quand le nombre de copies augmente fortement. Lors d'un examen, en effet, beaucoup de copies peuvent arriver en même temps, et la plateforme doit les corriger dans un délai raisonnable sans perdre en performance.

Ce travail cherche donc à savoir jusqu'à quel point l'architecture actuelle de CodEval peut absorber une charge croissante de copies, et quelles modifications dans l'organisation du traitement permettraient d'améliorer sa capacité sans rendre le système trop complexe à maintenir. Son objectif est d'évaluer la capacité de CodEval à monter en charge et de proposer des stratégies pour que la plateforme puisse servir un nombre croissant d'étudiants. Pour cela, nous décrivons d'abord le fonctionnement actuel, du lancement de la correction par l'enseignant jusqu'à l'affichage des résultats, puis nous mesurons ses limites par des tests de charge et nous recommandons enfin la solution qui offre le meilleur équilibre entre gain de performance et simplicité. Nous partons de trois hypothèses : le traitement actuel devient un goulot d'étranglement à partir d'un certain nombre de copies, avec un temps de correction qui grandit au même rythme que le nombre d'étudiants ; traiter plusieurs copies en même temps sur un seul serveur réduit nettement ce temps tant que le processeur et la mémoire ne sont pas saturés ; et ajouter plusieurs workers indépendants permet de suivre la charge presque proportionnellement sans changer en profondeur le code existant.

Pour les vérifier, nous lisons d'abord le code de CodEval, puis nous construisons un banc de test reproductible qui simule des évaluations avec un nombre variable d'étudiants et d'exercices, en mesurant le temps total de correction, le temps par copie et l'usage du processeur et de la mémoire, avant de comparer chaque configuration selon sa vitesse, ses ressources consommées et la complexité qu'elle ajoute. Le document s'organise en six chapitres : le premier présente le contexte et la problématique, le deuxième fait le point sur les travaux existants, le troisième décrit l'architecture proposée, le quatrième explique la méthode expérimentale, le cinquième donne les résultats des tests et le sixième les discute et formule des recommandations, avant une conclusion générale, les références bibliographiques et les annexes.

Chapitre 1 : Contexte et problématique

1.1 Évaluation automatique et CodEval

CodEval se présente sous la forme d'une application web composée de trois briques principales : une API construite avec le framework FastAPI, qui gère les utilisateurs, les évaluations et les soumissions ; un worker de correction, qui récupère les campagnes en attente dans la base de données PostgreSQL et les traite ; et une sandbox, qui exécute le code des étudiants dans un espace isolé avec des limites de temps et de mémoire pour garantir la sécurité du serveur. La figure 1.1 présente l'architecture générale de la plateforme.

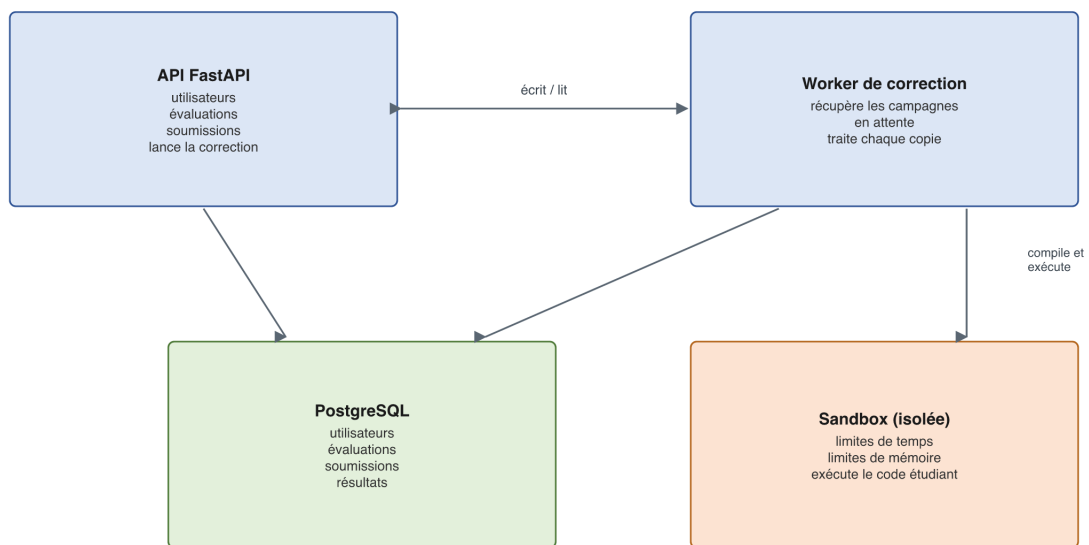


Figure 1.1 - Architecture générale de la plateforme CodEval

L'enseignant crée une évaluation, rédige ses exercices avec leurs jeux de tests, puis ouvre une session pour sa classe. Les étudiants rédigent leurs réponses dans un environnement contrôlé sans pouvoir exécuter leur code. Une fois la session fermée, l'enseignant lance la correction : le worker parcourt chaque participation, compile et exécute le code de chaque exercice contre les tests préparés, et attribue une note automatique.

CodEval prend en charge plusieurs types d'exercices : des exercices de code en C, des exercices d'algorithmique en pseudo-code (transpilés en Python pour être exécutés), des QCM, des questions à réponse courte, des grilles de correspondance et des vrai/faux. Pour les exercices de code, le barème combine des critères de déclaration (présence d'une fonction avec la bonne signature, déclaration d'une variable d'un type donné) et des tests d'exécution, chacun pesant ses points dans le total de l'exercice.

1.2 Problématique de la charge de travail

Aujourd'hui, le worker de CodEval traite les copies de manière strictement séquentielle : pour chaque participation, il itère sur les exercices un par un, compile, exécute et note avant de passer au suivant. Cette approche convient à de petits groupes, mais elle pose un problème de passage

à l'échelle. Lors d'un examen, plusieurs dizaines voire centaines d'étudiants soumettent leurs copies en même temps, et la plateforme doit les corriger dans un délai raisonnable.

Le temps de correction d'une copie dépend de la nature des exercices : un QCM ou un vrai/faux se corrige en quelques millisecondes, mais un exercice de code nécessite une compilation, une ou plusieurs exécutions en sandbox (chacune soumise à un délai maximal), et éventuellement la compilation de sondes pour vérifier les déclarations. Pour une évaluation comportant plusieurs exercices de code avec chacun plusieurs jeux de tests, le temps par copie peut atteindre plusieurs secondes. Multiplié par le nombre d'étudiants, le temps total de correction croît linéairement et peut devenir un goulot d'étranglement : les enseignants et les étudiants doivent alors attendre de longues minutes avant de consulter les résultats.

Par ailleurs, le traitement séquentiel sous-exploite les ressources du serveur. Pendant qu'une exécution en sandbox attend la fin de son délai, le processeur reste largement inactif.

À cette séquentialité s'ajoute une contrainte moins visible, mais qui pèse davantage : l'unité de travail du worker est la campagne entière. La fonction `claim_next` réserve une ligne `CorrectionRun`, c'est-à-dire une évaluation avec toutes ses participations, et la traite de bout en bout avant d'en réserver une autre. La granularité désigne précisément la taille de cette unité : ce qu'un correcteur prend en une fois. Pour une classe de soixante copies, la file ne contient donc qu'une seule tâche réservable, quel que soit le nombre de workers lancés : un seul trouve du travail, les autres interrogent une file vide et se rendorment. Ce plafond ne se lève pas en ajoutant des machines, car il ne vient pas du nombre de correcteurs mais de la taille du grain. C'est la raison pour laquelle le chapitre 3 redécoupe le travail avant de chercher à le distribuer.

1.3 Question de recherche et hypothèses

La question de recherche de ce travail est la suivante : jusqu'à quel point l'architecture actuelle de CodEval peut-elle absorber une charge croissante de copies, et quelles modifications dans l'organisation du traitement permettraient d'améliorer sa capacité sans rendre le système trop complexe à maintenir ?

Pour y répondre, nous formulons trois hypothèses :

- **H1** : le traitement séquentiel actuel devient un goulot d'étranglement à partir d'un certain nombre de copies, avec un temps total de correction qui croît linéairement avec le nombre d'étudiants.
- **H2** : traiter plusieurs copies en même temps sur un seul serveur (parallélisme intra-nœud) réduit significativement le temps de correction, tant que le processeur et la mémoire ne sont pas saturés.
- **H3** : ajouter plusieurs workers indépendants (parallélisme inter-nœuds) permet de suivre la charge de manière quasi proportionnelle sans modifier en profondeur le code existant.

L'objectif de ce travail est d'évaluer la capacité de CodEval à monter en charge et de proposer des stratégies pour que la plateforme puisse servir un nombre croissant d'étudiants. Pour vérifier ces hypothèses, nous étudierons le code source de CodEval, nous construirons un banc de test reproductible simulant des évaluations avec un nombre variable d'étudiants et d'exercices, et nous mesurerons le temps total de correction, le temps par copie ainsi que l'usage du processeur et de la mémoire pour chaque configuration testée.

Chapitre 2 : Systèmes étudiés

Ce chapitre présente les quatre architectures de traitement que nous avons tirées de la littérature pour améliorer la capacité de correction de CodEval : le traitement séquentiel, le parallélisme sur une machine (architecture A), les workers distribués autour d'un courtier de messages (architecture B) et la messagerie serverless élastique (architecture C). Chaque architecture est tirée d'une seule étude. Pour chacune, nous présentons d'abord cette étude (thème, auteurs, but, systèmes ou outils étudiés), puis le système que ses auteurs ont construit à la suite de leur travail, sa transposition à CodEval, son schéma, ses avantages et ses limites. Les limites portent sur ce système : celles que les auteurs énoncent eux-mêmes sont référencées, les autres découlent de notre analyse dans le contexte de CodEval. Nous terminons par une discussion sur les compromis identifiés.

2.1 Traitement séquentiel (architecture actuelle)

2.1.1 Description

L'étude. Étude sur le thème des systèmes de correction automatique de code, appelés « online judges », réalisée par Szymon Wasik, Maciej Antczak, Jan Badura, Artur Laskowski et Tomasz Sternal [1] et publiée en 2018 dans la revue *ACM Computing Surveys*, pour dresser l'état de l'art de ces systèmes.

But de l'étude. Classer les online judges selon leur objectif principal, en donner une définition formelle et résumer la méthode d'évaluation qu'ils ont en commun.

Systèmes étudiés.

- les plateformes de concours de programmation (par exemple Codeforces, ACM-ICPC Live Archive, Aizu Online Judge) ;
- les plateformes d'enseignement et de recrutement (par exemple CheckiO) ;
- les plateformes de défis de fouille de données ;
- les compilateurs en ligne (par exemple Ideone, Codepad) ;
- la plateforme Optil.io, proposée par les auteurs pour les problèmes d'optimisation.

Le système qu'ils ont construit. À partir de ce recensement, les auteurs définissent une procédure d'évaluation en trois étapes et la mettent en œuvre dans leur propre plateforme, Optil.io, présentée dans le même article comme l'online judge dont ils sont les auteurs et sur lequel ils ont le plus de contrôle [1]. Pour chaque soumission, le système collecte le code, le compile si nécessaire et vérifie que le binaire obtenu est exécutable ; il évalue ensuite ce binaire sur un ensemble de jeux de test dans un environnement d'exécution homogène et fiable, en vérifiant qu'il ne dépasse pas les limites de temps processeur, de mémoire vive et d'espace disque fixées ; il calcule enfin les notes obtenues sur chaque jeu de test [1]. L'utilisateur dépose sa solution par une interface web, sous forme de code source dans l'un des langages supportés. La façon la plus simple d'organiser cette procédure est de l'appliquer à une soumission, puis de passer à la suivante : c'est le traitement séquentiel.

Transposition à CodEval. C'est l'architecture actuellement en place. Un seul processus parcourt les copies une par une. Pour chaque participation, il itère sur les exercices, compile le code, exécute les tests en sandbox, puis enregistre le résultat en base de données avant de passer à la copie suivante. Chaque réponse suit donc exactement la procédure de Wasik et al. [1], et les réponses sont traitées l'une après l'autre.

2.1.2 Schéma d'architecture

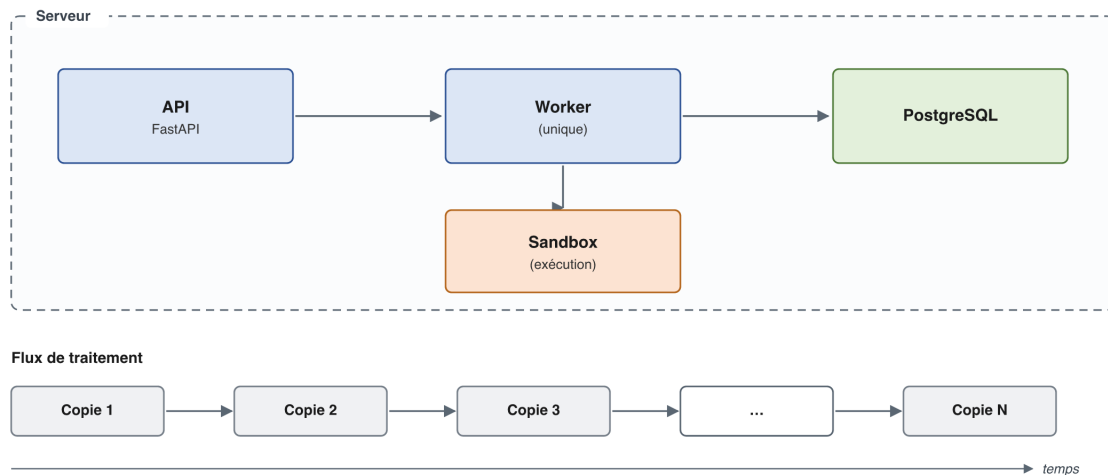


Figure 2.1 - Traitement séquentiel : organisation et déroulement

2.1.3 Avantages

- **Simplicité d'implémentation** : aucune gestion de concurrence, pas de synchronisation entre processus.
- **Facilité de débogage** : l'exécution est déterministe et les erreurs sont reproductibles dans l'ordre.
- **Infrastructure minimale** : un seul serveur suffit, sans composant supplémentaire.
- **Idempotence naturelle** : une copie déjà corrigée est simplement ignorée à la prochaine itération.

2.1.4 Limites

Limites que les auteurs énoncent eux-mêmes sur ce type de système :

- **La montée en charge est le point critique** : la scalabilité d'un système aussi interactif est cruciale, en particulier à l'approche de la date limite d'un concours, quand le nombre de soumissions augmente brutalement ; pour cette raison, ces systèmes sont déployés comme des services cloud de type PaaS et leur efficacité est obtenue par une architecture qui exploite la concurrence et le traitement parallèle [1].
- **Précision de la mesure du temps** : la limite de temps d'un jeu de test se compte souvent en millisecondes, et la méthode de mesure doit être assez sensible et déterministe pour distinguer d'aussi petites fractions de temps et rester reproductible d'une exécution à l'autre [1].

- **Sécurité de l'exécution** : le système exécute du code écrit par les utilisateurs, et ses concepteurs doivent le rendre résistant à un large éventail d'attaques [1].

Limites de la transposition à CodEval :

- **Temps de correction linéaire** : le temps total est proportionnel au nombre de copies ($T = N \times t_{\text{copie}}$).
- **Sous-utilisation des ressources** : pendant l'exécution en sandbox, le processeur reste inactif en attente du résultat.
- **Absence de tolérance aux pannes** : une erreur dans le worker arrête l'ensemble du processus de correction.
- **Non scalable** : impossible d'accélérer le traitement sans changer d'architecture.

2.2 Parallélisme sur une machine : pool de processus local (architecture A)

2.2.1 Description

L'étude. Étude sur le thème de la performance des online judges sur les machines multiprocesseurs, réalisée par Cheedoong Drung, Jianwen Wang et Ning Guo [8] et présentée en 2011 à la conférence internationale EMEIT (IEEE), pour augmenter le nombre de corrections qu'une même machine peut traiter en même temps.

But de l'étude. Améliorer la capacité d'un online judge installé sur une machine SMP (plusieurs processeurs partageant la même mémoire) sans perdre la précision de la mesure du temps d'exécution des programmes corrigés, qui sert à vérifier les limites de temps.

Outils et notions étudiés.

- une machine multiprocesseur symétrique, ou *Symmetric MultiProcessing* (SMP) ;
- un algorithme d'affinité, qui attache chaque programme en cours de correction à un processeur ;
- la théorie des files d'attente, pour modéliser l'arrivée et le traitement des soumissions.

Le système qu'ils ont construit. Les auteurs mettent en place un online judge fonctionnant dans un environnement SMP, dans lequel deux mécanismes sont ajoutés [8]. Le premier est l'algorithme d'affinité : chaque programme d'utilisateur en cours d'évaluation est attaché à un processeur, ce qui améliore la précision de la mesure de son temps de traitement. Le second est un dimensionnement fondé sur la théorie des files d'attente, qui fournit une analyse théorique des indicateurs de performance du système et permet de les améliorer. Avec ces mécanismes, les auteurs rapportent que la capacité moyenne de tâches corrigées simultanément augmente, que les verdicts rendus sont plus précis et que le nombre total d'ordinateurs nécessaires à l'online judge diminue, ce qui réduit le coût matériel [8].

Transposition à CodEval. La boucle séquentielle est remplacée par un pool de processus. Le worker principal distribue les participations à un `ProcessPoolExecutor` avec N workers, où N est configurable et borné par le nombre de cœurs CPU disponibles. Chaque worker du pool corrige une copie complète de manière indépendante, puis écrit ses résultats en base.

2.2.2 Schéma d'architecture

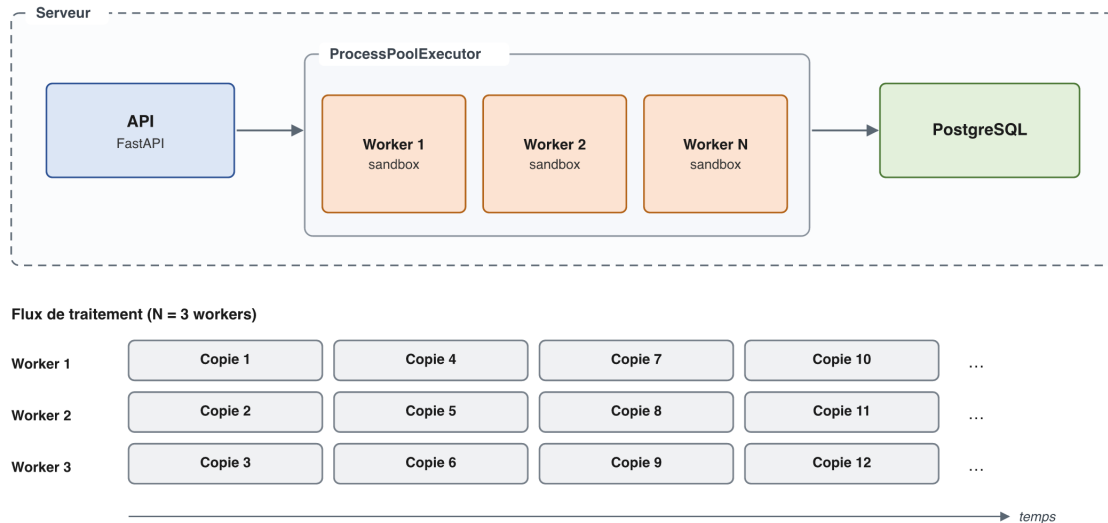


Figure 2.2 - Pool de processus local sur une machine multiprocesseur (architecture A), ici avec trois workers

2.2.3 Avantages

- **Accélération significative** : le temps de correction est divisé par N, le nombre de workers, tant que le processeur n'est pas saturé.
- **Déploiement simple** : aucune infrastructure supplémentaire requise, pas de file de messages ni de serveur additionnel.
- **Modifications limitées** : seule la boucle de traitement est modifiée, le reste du code reste inchangé.
- **Isolation des processus** : chaque worker s'exécute dans son propre processus, limitant l'impact d'un arrêt brutal à une seule copie.

2.2.4 Limites

Limites qui découlent du système décrit par les auteurs :

- **Le gain reste celui d'une seule machine** : la capacité de correction simultanée augmente, mais elle reste bornée par le nombre de processeurs de la machine SMP, et c'est précisément sur ce nombre de processeurs que repose le gain annoncé [8].
- **La précision des verdicts n'est pas gratuite** : corriger plusieurs programmes en même temps sur une même machine dégrade la mesure du temps d'exécution, et c'est pour corriger ce défaut que les auteurs doivent introduire l'algorithme d'affinité [8].

Limites de la transposition à CodEval :

- **Plafond de performance** : le gain est borné par le nombre de cœurs CPU du serveur, typiquement 4 à 16.
- **Contention sur la base de données** : plusieurs workers écrivant simultanément dans PostgreSQL peuvent créer des goulots d'étranglement sur les verrous de table.

- **Consommation mémoire** : chaque processus duplique l'espace mémoire du worker, ce qui limite le nombre de workers sur un serveur à ressources contraintes.
- **Pas de scalabilité horizontale** : impossible d'ajouter de la capacité au-delà des ressources d'un seul serveur.

2.3 Workers distribués autour d'un courtier de messages (architecture B)

2.3.1 Description

L'étude. Étude sur le thème des courtiers de messages dans les architectures de microservices, réalisée par Ahmed Gamal Ibrahim, Rui Pedro Lopes, José Rufino et Paulo Leitão [3] et présentée à la conférence OL2A 2025 (Springer), pour déterminer quel courtier assure une communication efficace, fiable et capable de monter en charge entre des services.

But de l'étude. Comparer les courtiers les plus répandus lorsque des microservices se coordonnent par échange de messages, sans chef d'orchestre central : on parle de « chorégraphie », chaque service réagissant aux messages qu'il reçoit. Les auteurs rappellent qu'une communication défaillante ou retardée peut empêcher l'analyse des données en temps réel.

Outils étudiés.

- Apache Kafka ;
- ActiveMQ Artemis ;
- RabbitMQ ;
- NATS ;
- des clients écrits en Java (Spring Boot) et en Python.

Ce que l'étude établit. Cette étude ne livre pas un système de production : elle compare des outils. Nous la conservons parce que cette comparaison est celle qui fonde, preuves à l'appui, le choix du courtier au cœur de l'architecture B. Les auteurs placent des services producteurs et consommateurs dans un environnement d'expérimentation standardisé et les font communiquer à travers chacun des quatre courtiers tour à tour, avec des clients écrits en Java (Spring Boot) et en Python, en mesurant la latence, le débit, la capacité à monter en charge et la fiabilité [3]. Leurs résultats : Kafka offre de bonnes performances en traitement temps réel, avec une faible latence et une grande fiabilité ; ActiveMQ Artemis est fiable mais présente une latence nettement plus élevée ; RabbitMQ obtient une latence compétitive mais rencontre des difficultés lors des coupures réseau ; NATS, conçu pour la faible latence et le débit élevé, montre une excellente montée en charge et un excellent débit dans tous les scénarios [3]. C'est donc le courtier, et non le principe de la file, qui détermine la latence et la fiabilité réellement obtenues.

Transposition à CodEval. Un courtier de messages est placé entre l'API et les workers. Lorsqu'une correction est lancée, l'API joue le rôle de service producteur : elle découpe la campagne en tâches unitaires (une par participation) et les publie dans le courtier. Des workers indépendants, potentiellement déployés sur des serveurs différents, jouent le rôle de services consommateurs : ils retirent les tâches du courtier et corrigent les copies en parallèle, puis écrivent les résultats dans la base partagée. Le courtier retenu est l'un des quatre comparés par

l'étude ; la structure de l'architecture ne change pas d'un courtier à l'autre, seules la latence et la fiabilité mesurées varient [3].

2.3.2 Schéma d'architecture

Le schéma ci-dessous reproduit le dispositif mis en place par les auteurs : des services producteurs et des services consommateurs, écrits en Java (Spring Boot) et en Python, communiquent uniquement par échange de messages à travers un courtier, dans un environnement d'expérimentation standardisé où sont mesurés la latence, le débit, la montée en charge et la fiabilité [3]. Le bloc central est occupé tour à tour par Apache Kafka, ActiveMQ Artemis, RabbitMQ puis NATS : l'architecture est identique pour les quatre courtiers, seul le courtier placé au centre change d'une série de mesures à l'autre.

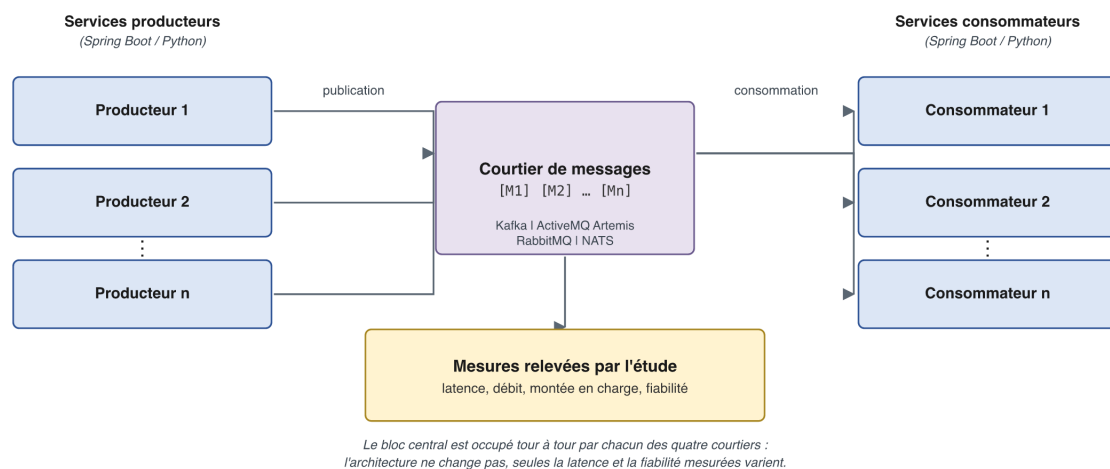


Figure 2.3 - Dispositif à courtier de messages mesuré par Ibrahim et al. (architecture B)

2.3.3 Avantages

- **Scalabilité horizontale** : ajouter un serveur ajoute proportionnellement de la capacité de traitement. Le gain est quasi linéaire avec le nombre de workers.
- **Tolérance aux pannes** : si un worker tombe en panne, ses tâches non acquittées sont remises dans la file et reprises par un autre worker. La correction continue sans interruption.
- **Élasticité** : des workers peuvent être lancés ou arrêtés dynamiquement en fonction de la charge.
- **Découplage** : l'API et les workers sont indépendants. L'API peut accepter de nouvelles campagnes même si les workers sont temporairement saturés.

2.3.4 Limites

Limites établies par l'étude elle-même :

- **Le comportement de l'architecture dépend entièrement du courtier retenu** : la latence varie fortement d'un courtier à l'autre, ActiveMQ Artemis étant nettement plus lent que les autres [3].
- **La fiabilité annoncée n'est pas acquise** : RabbitMQ, malgré une latence compétitive, rencontre des difficultés lors des coupures réseau [3], c'est-à-dire au moment précis où la tolérance aux pannes est attendue d'une architecture à file de messages.

- **Les mesures sont faites hors contexte éducatif** : l'étude évalue des microservices échangeant des messages à des débits sans rapport avec ceux d'une campagne de correction, et ne dit rien du coût de l'exécution en sandbox ni de l'écriture des notes en base.

Limites de la transposition à CodEval :

- **Complexité opérationnelle** : le déploiement nécessite un courtier de messages à installer et superviser, un suivi des workers, et une gestion des échecs (tâches non acquittées, workers en dépassement de délai).
- **Infrastructure supplémentaire** : chaque worker additionnel est un serveur à provisionner, configurer et maintenir, ce qui augmente les coûts d'hébergement.
- **Latence réseau** : la communication entre les workers et la base de données introduit une latence supplémentaire par rapport à un accès local.
- **Contention sur la base de données** : avec de nombreux workers écrivant simultanément, PostgreSQL peut devenir le goulot d'étranglement si la base n'est pas dimensionnée en conséquence.
- **Modifications significatives du code** : l'intégration d'une file de messages nécessite de repenser la logique de lancement des corrections et la gestion du cycle de vie des tâches.

2.4 Messagerie serverless élastique (architecture C)

2.4.1 Description

L'étude. Étude sur le thème de la messagerie « serverless » à grande échelle, réalisée par Juntao Ji, Yubao Fu, Rongtong Jin et Qingshan Lin [5] (Alibaba Cloud) et présentée en 2025 dans la sélection industrielle de la conférence FSE (ACM), pour offrir un service de messages dont la capacité suit la croissance du trafic sans limite.

But de l'étude. Faire fonctionner le logiciel de messagerie en mode serverless, c'est-à-dire sans que l'utilisateur gère le moindre serveur, avec une capacité qui s'ajuste seule à la demande, tout en restant compatible avec les protocoles et les clients que les applications utilisent déjà.

Outils et systèmes étudiés.

- Apache RocketMQ, le socle de l'architecture ;
- les protocoles RabbitMQ, MQTT et Kafka, réimplémentés au-dessus de ce socle ;
- RabbitMQ open source, qui sert de point de comparaison.

Le système qu'ils ont construit. Les auteurs ont construit, et exploitent commercialement sur Alibaba Cloud, une architecture de messagerie serverless bâtie sur Apache RocketMQ, au-dessus de laquelle ils ont réimplémenté les solutions de messagerie les plus répandues : RabbitMQ, MQTT et Kafka [5]. Elle repose sur les éléments suivants :

- **Séparation du stockage et du calcul.** Une couche d'accès sans état reçoit les messages, quel que soit le protocole, et une couche de stockage distincte les conserve. Chaque couche grossit indépendamment de l'autre, ce qui convient à des charges imprévisibles.
- **Partitions d'écriture élastiques,** qui suppriment la limite de débit d'une file unique.

- **Files légères**, qui permettent d'avoir des millions de files avec un temps de démarrage à froid minimal.
- **Un RabbitMQ reconstruit sur RocketMQ**, pris comme cas d'étude parce que son architecture d'origine est difficile à faire monter en charge. Il reste compatible avec tous les clients open source et n'impose plus de limite de débit par file ; les auteurs annoncent des capacités de gestion des métadonnées de files et d'accumulation de messages supérieures de plus de 1 000 % à celles de RabbitMQ open source [5].

Transposition à CodeEval. La file n'est plus un serveur que l'établissement installe : elle est confiée à un service de messagerie serverless géré par un fournisseur cloud. L'API y dépose une tâche par copie. Des workers sans état, lancés à la demande sous forme de conteneurs éphémères, consomment la file : leur nombre suit la profondeur de la file et redescend à zéro quand il n'y a plus rien à corriger. Les notes sont toujours écrites dans PostgreSQL.

2.4.2 Schéma d'architecture

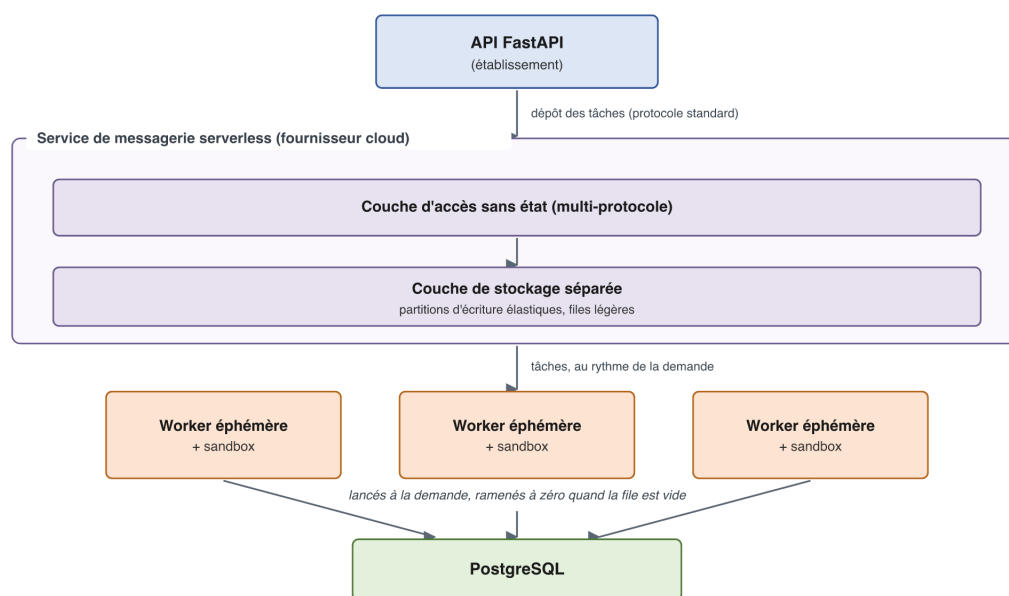


Figure 2.4 - Messagerie serverless élastique et workers éphémères (architecture C)

2.4.3 Avantages

- **Élasticité complète** : la capacité de correction passe de zéro à un grand nombre de workers selon la charge, sans intervention.
- **Aucun serveur de file à administrer** : le fournisseur gère la disponibilité, la reprise et la montée en charge de la messagerie.
- **Pas de limite de débit** par file, grâce à la séparation du stockage et du calcul.
- **Paieement à l'usage** : en dehors des périodes d'examen, les workers ne consomment rien.

2.4.4 Limites

Limites qui découlent du système décrit par les auteurs :

- **Un service commercial, exploité par un fournisseur** : l'architecture est celle d'un produit d'Alibaba Cloud [5]. L'établissement ne l'installe pas, il la loue ; la correction s'arrête si le service ou la connexion Internet tombe. La compatibilité avec les clients open source revendiquée par les auteurs [5] réduit, sans l'éliminer, le risque de verrouillage chez le fournisseur.
- **Une échelle disproportionnée** : le système est dimensionné pour des millions de files et pour des capacités d'accumulation plus de dix fois supérieures à celles de RabbitMQ open source [5], alors qu'une campagne de CodEval produit quelques centaines de messages.

Limites de la transposition à CodEval :

- **Données des étudiants hors de l'établissement** : les copies transitent par un service tiers, ce qui pose des questions de protection des données.
- **Démarrage à froid des workers** : le soin apporté par les auteurs au démarrage à froid porte sur les files, pas sur nos workers ; chaque worker éphémère doit charger le compilateur et la sandbox avant de corriger, ce qui ajoute de la latence au début de chaque campagne.
- **Sandbox à adapter** : l'isolation doit fonctionner dans l'environnement d'exécution imposé par le fournisseur.
- **Coût récurrent et impossibilité de reproduire l'architecture** sur le matériel dont nous disposons.

2.5 Discussion sur les limites

Les quatre architectures présentent des compromis distincts entre simplicité et performance.

Traitement séquentiel

- Adapté aux petites cohortes, moins de 30 étudiants, où le temps de correction reste acceptable, de l'ordre de quelques minutes.
- Au-delà, le temps linéaire freine l'expérience utilisateur : un enseignant lançant la correction de 200 copies avec 5 exercices chacune peut attendre plus de 30 minutes avant d'obtenir les résultats.

Pool de processus (architecture A)

- Résout partiellement le problème en divisant le temps par le nombre de cœurs disponibles.
- Atteint rapidement son plafond : un serveur typique dispose de 4 à 8 cœurs, ce qui limite le gain maximal à un facteur 4 à 8.
- La contention sur PostgreSQL peut dégrader les performances au-delà de 4 workers concurrents, ce qui réduit le gain réel en dessous du gain théorique.

Architecture distribuée (architecture B)

- Offre une scalabilité théoriquement illimitée, mais au prix d'une complexité opérationnelle considérable.
- Pour une plateforme éducative comme CodEval, déployée dans un contexte universitaire avec des ressources d'infrastructure limitées, cette complexité peut constituer un obstacle majeur à la maintenance et à l'évolution du système.

Serverless élastique (architecture C)

- Pousse la logique distribuée jusqu'au bout : supprime l'administration de la file et ajuste la capacité à la charge.
- Déplace le problème vers la dépendance à un fournisseur cloud, la sortie des copies de l'établissement et un coût récurrent.
- Est conçue pour des volumes sans commune mesure avec ceux de CodEval.

Limites communes

- **Contention sur la base de données (A et B) :** lorsque plusieurs workers écrivent simultanément les résultats de correction, les verrous de PostgreSQL peuvent créer un goulot d'étranglement qui annule une partie du gain obtenu par le parallélisme. Pantelic et al. [6] (*Future Internet*, 2026) montrent, dans un banc d'essai conteneurisé sur un seul nœud, que les performances de la couche de persistance d'une architecture de microservices dépendent fortement de la composition de la charge et du niveau de concurrence : une base SQL indexée donne les meilleurs résultats sur les charges dominées par les lectures, alors que sur les charges dominées par les écritures, une base orientée documents (MongoDB) obtient une latence p95 plus faible et un débit plus élevé. La correction étant une charge dominée par les écritures, une note par réponse, ce point doit être mesuré dans notre contexte.
- **Tolérance aux pannes :** c'est un critère discriminant.
 - Le traitement séquentiel et le pool de processus partagent la même vulnérabilité : un arrêt brutal du serveur arrête l'ensemble de la correction.
 - L'architecture B offre une résilience réelle, au prix de mécanismes de supervision et de retraitement des tâches échouées à mettre en place.
 - L'architecture C offre aussi une résilience réelle, en confiant ces mécanismes au fournisseur.

Le chapitre suivant part de ces quatre architectures, de leurs avantages et de leurs limites, pour proposer une architecture adaptée à CodEval.

Chapitre 3 : Architecture proposée

3.1 Bilan des architectures étudiées

Le tableau suivant résume ce que chaque architecture apporte et ce qui lui manque.

Architecture	Ce qu'elle apporte	Ce qui lui manque
Séquentielle	Simplicité, idempotence, aucun composant en plus	Temps linéaire, processeur sous-utilisé, pas de reprise
Pool local (A)	Exploite les cœurs d'un serveur, peu de code modifié	Plafond d'un seul serveur, un arrêt brutal stoppe la campagne
Distribuée (B)	Ajout de serveurs, reprise des tâches d'un worker mort	Courtier à déployer et à superviser, code à refondre
Serverless (C)	Capacité ajustée à la charge, aucun serveur de file	Fournisseur cloud, copies hors de l'établissement, démarrage à froid, coût

Tableau 3.1 - Bilan des architectures étudiées

Trois constats traversent ces architectures :

- **Le moteur de correction n'est pas en cause.** La fonction `grade_exercise` corrige une réponse de façon indépendante des autres. Ce qui limite CodEval, c'est la façon dont les copies sont distribuées aux processus, pas la façon dont chacune est corrigée.
- **La base de données est déjà partagée par tous.** Toutes les architectures lisent les copies et écrivent les notes dans PostgreSQL. B et C ajoutent un second point de coordination, un courtier ou un service cloud, alors que PostgreSQL sait déjà jouer ce rôle : le worker actuel réserve ses campagnes avec la clause `SELECT ... FOR UPDATE SKIP LOCKED`.
- **Les copies n'ont pas toutes le même coût.** Un QCM se corrige en quelques millisecondes, une réponse en C correcte en quelques centaines de millisecondes, mais une boucle infinie épuise le délai de chaque test, soit 2 secondes par test dans CodEval et donc 12 secondes pour les 6 tests de l'exercice « Calculatrice en boucle ». Distribuer des copies entières laisse donc des workers inactifs pendant qu'un autre termine une copie lente.

3.2 Des limites aux exigences de conception

Chaque limite relevée au chapitre 2 devient une exigence que l'architecture proposée doit satisfaire. La figure 3.1 montre cette correspondance.

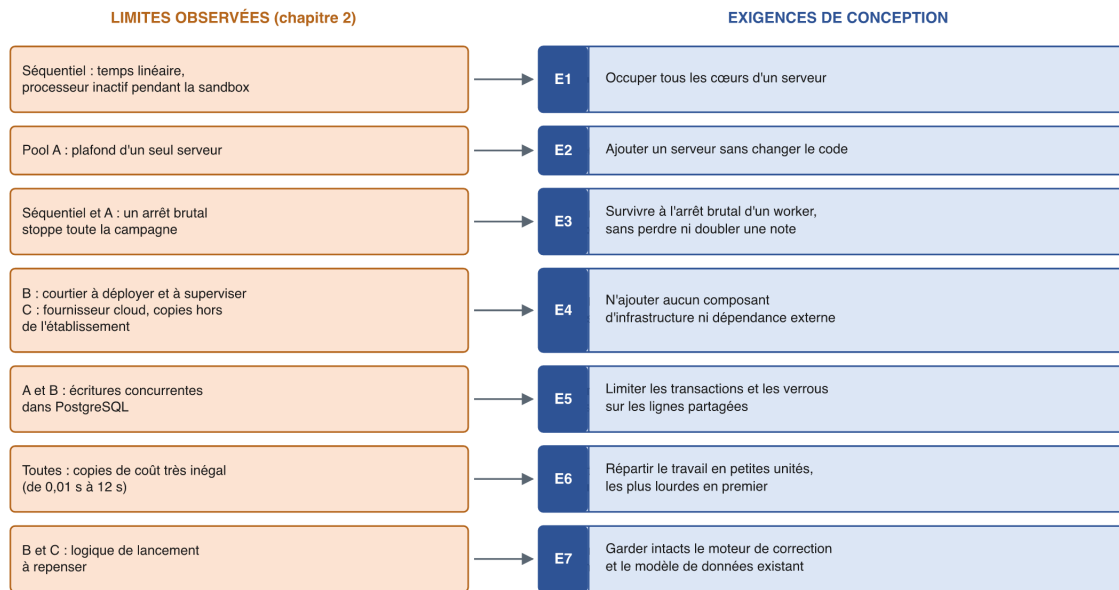


Figure 3.1 - Traduction des limites des architectures étudiées en exigences de conception

3.3 Principe de l'architecture proposée

3.3.1 Ce que fait CodEval aujourd'hui

Dans l'architecture actuelle, lancer une correction crée une campagne dans la table `correction_runs`. Un processus unique, le worker, réserve cette campagne, charge la liste des participations, puis les parcourt une par une : pour chaque copie il itère sur les exercices, appelle `grade_exercise`, compile et exécute le code en sandbox quand l'exercice l'exige, enregistre la note, puis passe à la copie suivante. Le travail restant n'existe nulle part ailleurs que dans la position courante de cette boucle, en mémoire du processus. C'est de là que viennent les trois faiblesses relevées au chapitre 1 : le temps total croît linéairement avec le nombre de copies, le processeur reste inactif pendant que la sandbox attend l'expiration de ses délais, et l'arrêt du processus fait perdre la totalité de la progression de la campagne.

3.3.2 L'idée de l'architecture proposée

L'architecture proposée, notée **P** dans la suite, change une seule chose de fond : **le travail restant cesse d'être une position dans une boucle et devient un ensemble de lignes persistantes dans la base de données**. La campagne n'est plus parcourue, elle est d'abord entièrement découpée en unités de travail élémentaires, puis ces unités sont déposées dans une file ; des processus correcteurs interchangeables viennent y puiser tant qu'il reste quelque chose à faire.

Concrètement, le service proposé fonctionne ainsi. Lorsque l'enseignant lance une correction, l'API ne se contente plus de créer la campagne : dans la même transaction, elle énumère tous les couples (copie, exercice) de l'évaluation et insère une ligne par couple dans une nouvelle table `correction_tasks`, chacune à l'état `pending` et porteuse d'une priorité calculée à partir du type d'exercice. La campagne est donc, dès sa création, intégralement décrite sous forme de travail à faire, visible par n'importe quel processus connecté à la base.

À partir de là, chaque processus correcteur exécute en boucle le même cycle, sans jamais parler aux autres : il réserve un petit lot de tâches `pending` dans la file, les marque `running` en y

inscrivant un bail, corrige chaque réponse en appelant le moteur `grade_exercise` existant, puis écrit les notes obtenues et marque les tâches done en une seule transaction. Il recommence jusqu'à ce que la file ne contienne plus de tâche réservable pour la campagne. La campagne est terminée lorsque le nombre de tâches done égale le nombre de tâches créées au départ. Comme les processus ne se coordonnent qu'à travers la file, on peut en lancer plusieurs sur un même serveur, et lancer la même commande sur un second serveur pointant vers la même base, sans modifier une ligne de code.

3.3.3 Ce qui est ajouté par rapport à l'architecture actuelle

Cinq éléments sont ajoutés, chacun répondant à une limite identifiée.

1. Une table-file `correction_tasks`. Elle stocke le travail restant dans PostgreSQL au lieu de la mémoire du worker. Chaque ligne porte un état (`pending`, `running`, `done`, `failed`), une priorité, un bail et un compteur de tentatives. Les correcteurs y réservent leurs tâches avec `SELECT ... FOR UPDATE SKIP LOCKED`, déjà utilisé par le worker actuel, ce qui empêche deux processus de prendre la même tâche. Elle joue le rôle du courtier de l'architecture B, sans ajouter d'infrastructure (E4, E2).

2. Le découpage en tâches fines, ordonnées par coût estimé. L'unité de travail devient la réponse à un exercice et non la copie. Les tâches sont servies des plus lourdes aux plus légères : code C, algorithmique, questions courtes, puis QCM et vrai/faux. Cela évite qu'une tâche longue commencée en fin de campagne fasse attendre tous les autres correcteurs (E6).

3. Un superviseur multi-processus sur chaque serveur. `app.worker --processes N` lance et surveille N correcteurs sur une machine, pour occuper les cœurs pendant qu'un processus attend la sandbox (E1). Contrairement à l'architecture A, ils prennent leur travail dans la file : le même binaire lancé sur un second serveur fonctionne à l'identique (E2).

4. Un bail sur chaque tâche réservée. Le correcteur inscrit une date d'expiration et la prolonge tant qu'il est vivant. S'il meurt, le bail expire et la tâche redevient `pending`. Le compteur de tentatives bascule en `failed` une tâche qui échoue de façon répétée. Avec la contrainte d'unicité sur (campagne, copie, exercice), qui interdit une double note après reprise, un arrêt brutal coûte au plus les tâches en cours (E3).

5. L'écriture par lot et la suppression du compteur partagé. Les notes et changements d'état d'un lot de k tâches sont écrits en une seule transaction. Le compteur `processed` de `correction_runs`, point de contention si tous les processus l'incrémentent, n'est plus écrit : la progression se lit en comptant les tâches done (E5).

3.4 Description de l'architecture

3.4.1 Vue d'ensemble

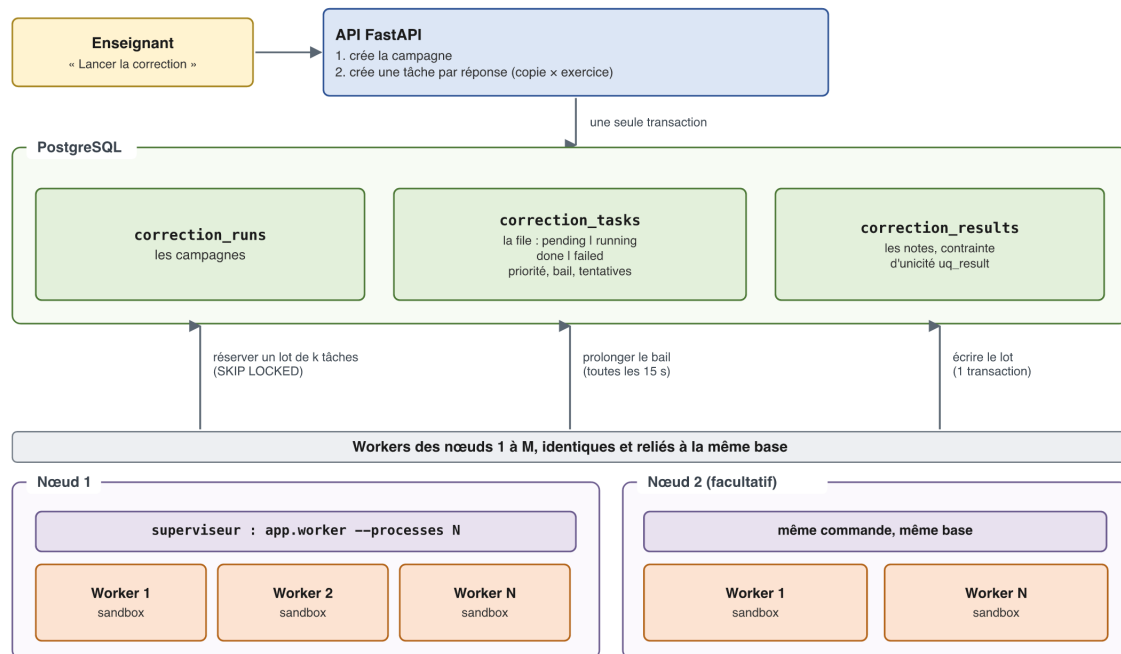


Figure 3.2 - Vue d'ensemble de l'architecture proposée (P)

3.4.2 Composants

L'architecture compte cinq éléments, dont un seul est nouveau.

L'API FastAPI. Elle reçoit la demande de l'enseignant et, dans la transaction qui crée la campagne, insère une tâche par réponse à corriger. Son rôle s'arrête là : elle ne corrige rien et n'attend pas la fin de la campagne.

La file `correction\_tasks`. C'est le seul composant nouveau, et c'est une simple table PostgreSQL. Elle contient le travail restant, une ligne par réponse, et sert de point de rendez-vous unique entre tous les workers, qui ne communiquent jamais directement entre eux.

Le superviseur. Sur chaque nœud, la commande `app.worker --processes N` lance un processus parent, le superviseur, qui crée N workers, les surveille et en relance un s'il meurt. Il ne corrige aucune réponse et ne distribue aucun travail : il ne lit même pas la file, à la différence du répartiteur local de l'architecture A.

Le worker. C'est le processus qui corrige. Il réserve lui-même un lot de tâches dans la file, appelle `grade_exercise` pour chacune, écrit les notes, et recommence tant qu'il reste des tâches réservables. Tous les workers sont identiques et interchangeables, quel que soit le nœud qui les héberge.

La sandbox. Elle reste le mécanisme actuel d'exécution isolée du code étudiant, inchangé. Chaque worker lance la sienne et attend sa fin ; c'est pendant cette attente que les autres workers du nœud occupent les cœurs.

3.4.3 Unité de travail et granularité

La **granularité** désigne la taille de l'unité de travail qu'un correcteur réserve en une fois. Elle détermine le nombre de tâches présentes dans la file pour une même campagne, donc le nombre de correcteurs qui peuvent y travailler en parallèle : un grain grossier laisse des workers inactifs

faute de tâche à prendre, un grain fin les occupe tous mais multiplie les accès à la base. Choisir la granularité, c'est arbitrer entre ces deux coûts. L'architecture actuelle retient le grain le plus grossier possible, la campagne (section 1.2), ce qui plafonne le parallélisme à un seul worker.

L'unité de travail est la **réponse** : un couple (copie, exercice). Ce choix répond à l'exigence E6. Avec un découpage par copie, une copie contenant une boucle infinie occupe un worker pendant plus de 12 secondes et retarde la fin de la campagne, alors que les autres workers ont déjà fini. Avec un découpage par réponse et les tâches lourdes en tête de file, cette réponse lente démarre tôt et les réponses rapides comblent les trous.

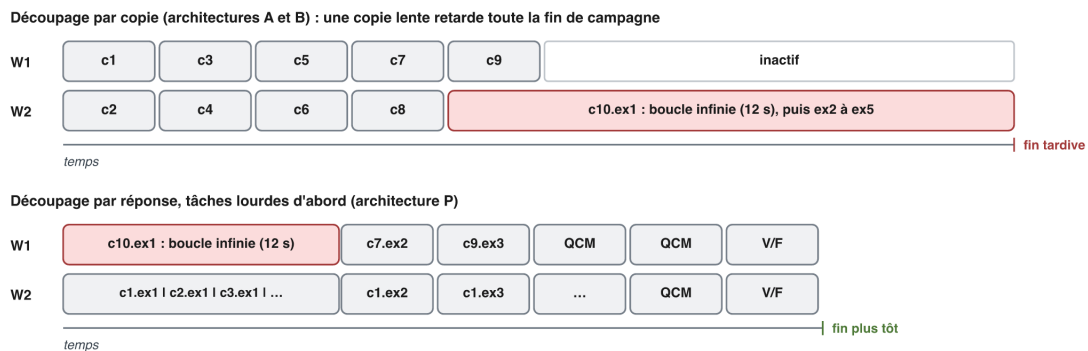


Figure 3.3 - Effet de la granularité sur l'équilibre de charge (c = copie, ex = exercice)

Une granularité plus fine a un coût : chaque tâche demande une réservation et une écriture en base. Pour l'amortir, un worker réserve et écrit ses tâches **par lots de k**, avec k valant 4 par défaut et réglable. Le chapitre 4 prévoit une mesure qui compare le découpage par copie et le découpage par réponse pour vérifier ce choix.

La priorité d'une tâche est fixée à sa création selon le type d'exercice : 3 pour le code C (compilation et exécution), 2 pour l'algorithmique (transpilation et exécution), 1 pour les QCM, vrai/faux, correspondances et réponses courtes (comparaison directe).

3.4.4 Cycle de vie d'une tâche

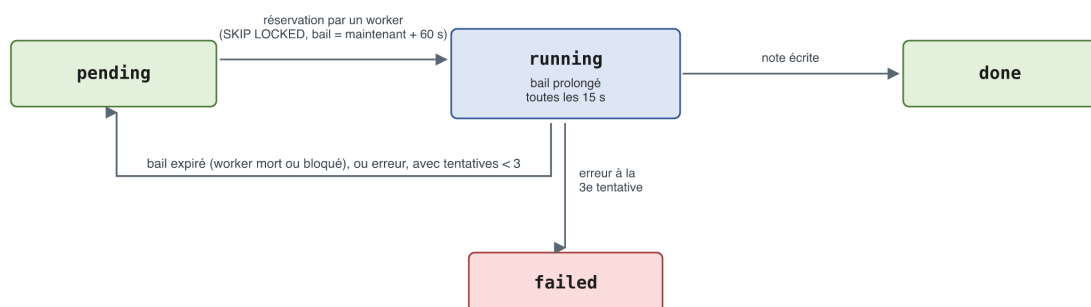


Figure 3.4 - États d'une tâche de correction

Une tâche failed ne bloque pas la campagne : celle-ci se termine avec le statut partial, comme aujourd'hui quand process_run rencontre une exception sur un exercice. L'enseignant voit les réponses concernées et peut relancer une campagne.

3.4.5 Déroulement d'une campagne

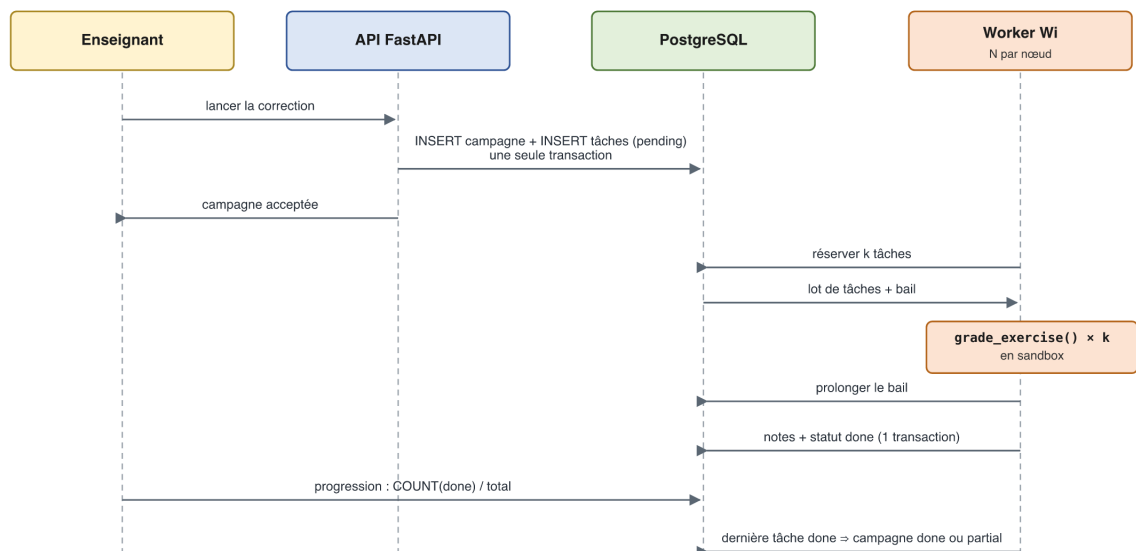


Figure 3.5 - Séquence de traitement d'une campagne

3.4.6 Déploiement

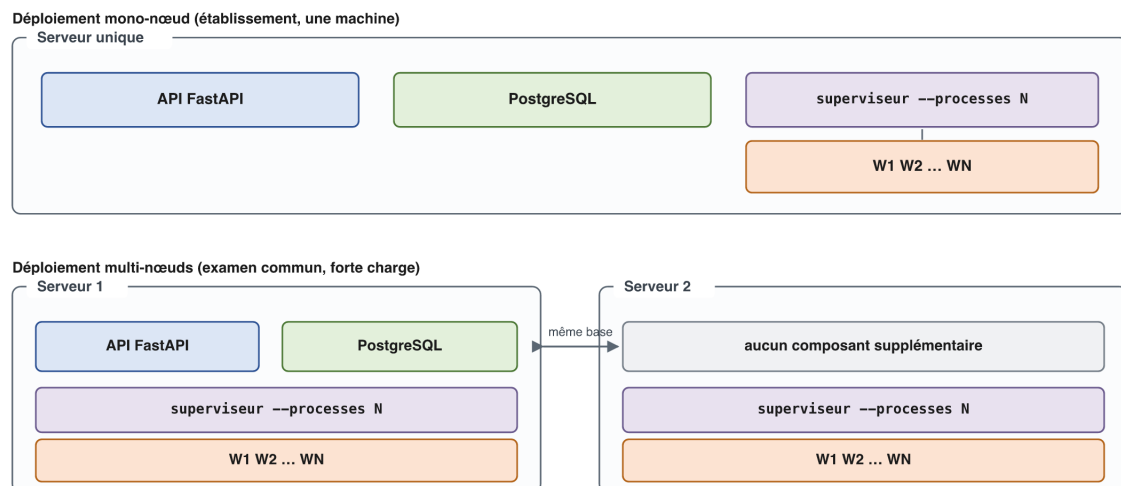


Figure 3.6 - Les deux modes de déploiement de l'architecture proposée

3.5 Comparaison avec les architectures étudiées

Critère	Séquentielle	Pool local (A)	Distribuée (B)	Serverless (C)	Proposée (P)
Étude d'origine	Wasik et al. [1]	Drung et al. [8]	Ibrahim et al. [3]	Ji et al. [5]	Ce travail
Parallélisme sur un serveur	Non	Oui (N processus)	Oui (M workers)	Oui (workers éphémères)	Oui (N processus)
Ajout de serveurs	Non	Non	Oui	Automatique	Oui
Composants à déployer	API, PostgreSQL	API, PostgreSQL	API, PostgreSQL, courtier	API, PostgreSQL, service cloud	API, PostgreSQL
Données hors de l'établissement	Non	Non	Non	Oui	Non
Unité de travail	Campagne	Copie	Copie	Copie	Réponse (lot de k)

Critère	Séquentielle	Pool local (A)	Distribuée (B)	Serverless (C)	Proposée (P)
Reprise après arrêt d'un worker	Non (campagne bloquée)	Non (campagne bloquée)	Oui (file de reprise)	Oui (gérée par le service)	Oui (bail de 60 s)
Écritures sur la ligne de campagne	1 par copie	1 par copie	1 par copie	1 par copie	Aucune (progression calculée)
Modifications du code	Aucune	Modérées	Importantes	Importantes	Modérées

Tableau 3.2 - Comparaison de l'architecture proposée avec les architectures étudiées

L'architecture proposée a aussi ses limites, qu'il faut garder en vue :

- **La file coûte des écritures.** Chaque tâche ajoute une insertion et deux mises à jour. Au rythme de CodEval, quelques dizaines de réponses corrigées par seconde au plus, cette charge reste faible ; mais au-delà de quelques dizaines de workers, la réservation elle-même pourrait devenir un point d'attente. À cette échelle, un courtier dédié comme dans l'architecture B redeviendrait pertinent.
- **L'attente active.** Un worker sans tâche interroge la base toutes les 2 secondes. Le premier lot d'une campagne peut donc attendre jusqu'à 2 secondes. Le mécanisme de notification de PostgreSQL pourrait réveiller les workers dès la création des tâches.

Le chapitre suivant décrit l'environnement, les métriques, les critères de réussite et le protocole qui permettent de tester cette hypothèse en comparant les architectures A, B et P à la référence séquentielle. L'architecture C n'est pas mesurée ; le chapitre 4 explique pourquoi.

Chapitre 4 : Méthodologie expérimentale

Ce chapitre explique comment nous comparons quatre façons de corriger : le traitement séquentiel actuel, qui sert de référence, les architectures A et B, et l'architecture proposée P.

L'ordre est le suivant. Le matériel et les logiciels d'abord. Les copies à corriger ensuite, et la façon de les construire. Puis ce que nous mesurons, avec quels indicateurs et quels seuils. Le reste est un mode d'emploi : le protocole complet est décrit sur l'architecture actuelle, et chaque architecture suivante n'ajoute que ce qui change. Les résultats sont au chapitre 5.

4.1 Matériel et logiciels

4.1.1 Poste de mesure

Toutes les mesures se font sur un seul poste, celui du développement de CodEval.

Composant	Spécification
Machine	MacBook Pro 14 pouces, modèle Mac15,3
Processeur	Apple M3, 8 cœurs : 4 cœurs de performance et 4 cœurs d'efficacité
Mémoire	8 Go de mémoire unifiée
Stockage	SSD interne
Système	macOS 26.6
Alimentation	Sur secteur pendant toute la campagne de mesures

Tableau 4.1 - Poste de mesure

Les deux types de cœurs n'ont pas la même puissance, et macOS choisit seul où placer chaque processus. Au-delà de 4 workers, les processus en trop tombent en partie sur les cœurs d'efficacité. Le gain par worker ajouté devrait donc baisser à partir de 4, et les mesures doivent le montrer.

4.1.2 Logiciels

Logiciel	Version	Rôle
Python	3.12.13 (environnement du backend)	Exécution de CodEval, des workers et du banc
FastAPI, SQLAlchemy, pycpg	Versions de requirements.txt, figées par pip freeze	Backend de CodEval
PostgreSQL	18.3 (Postgres.app)	Base de données, et file de tâches pour P
Apple clang	21.0.0 (appelé sous le nom gcc)	Compilation des réponses en C
Docker Desktop	29.2.1	Redis pour B, nœuds émulés pour l'expérience X3
Redis	7.2 (image redis:7.2-alpine)	File de messages de l'architecture B uniquement
psutil	Dernière version stable, figée	Échantillonnage du processeur et de la mémoire

Logiciel	Version	Rôle
CodEval	Commit de référence, empreinte enregistrée	Code testé

Tableau 4.2 - Logiciels utilisés et leur rôle

La base de test `codeval_bench` est séparée de la base de développement. PostgreSQL garde sa configuration par défaut : ses 100 connexions suffisent aux 8 workers, à l'orchestrateur et à l'échantillonneur.

4.1.3 Instruments de mesure

Quatre outils entourent l'application pendant chaque mesure. Ils observent la correction, ils ne la font pas.

- **L'orchestrateur** prépare la base, démarre les workers, déclenche la campagne, la chronomètre et exporte les données brutes à la fin.
- **L'échantillonneur** relève toutes les 500 millisecondes le processeur et la mémoire des workers et de leurs processus fils, et toutes les secondes le nombre de sessions en attente de verrou dans PostgreSQL.
- **La sonde d'API** interroge l'API toutes les 200 millisecondes pendant la correction et note son temps de réponse.
- **L'horloge de référence** est celle de PostgreSQL. Dans la base de test, chaque note enregistrée porte l'heure de son écriture. Toutes les durées en découlent, ce qui évite de comparer des horloges différentes.

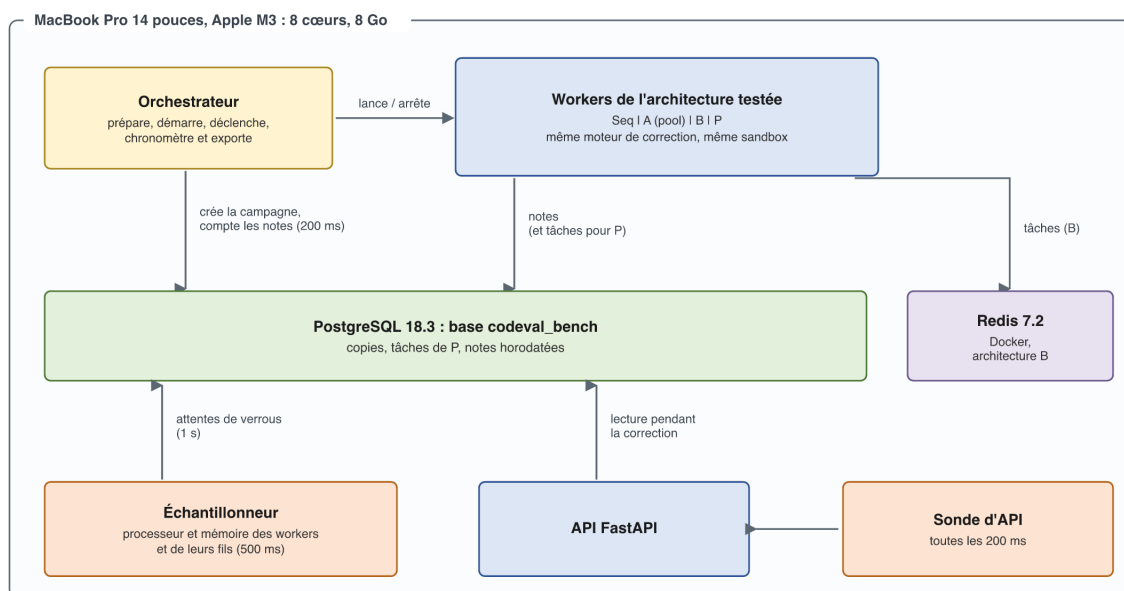


Figure 4.1 - Dispositif expérimental sur le poste de mesure

4.1.4 Conditions de mesure

Avant chaque série de mesures :

1. brancher le poste sur secteur et désactiver le mode économie d'énergie ;

2. fermer toutes les applications sauf le terminal, et suspendre l'indexation Spotlight ;
3. empêcher la mise en veille pendant la série ;
4. vérifier l'absence de bridage thermique ;
5. entre deux mesures, attendre 60 secondes pour que le processeur revienne à sa température de repos.

Le bridage thermique est la baisse de fréquence que le système décide lui-même quand la puce chauffe trop, pour la protéger. Le processeur travaille alors plus lentement, sans aucun message d'erreur. Une mesure lancée sur une machine encore chaude serait donc ralentie pour une raison étrangère à l'architecture testée.

4.2 Données corrigées pendant les mesures

Les données de test viennent de contenus de CodEval, pas d'exercices inventés pour le banc. L'application les charge elle-même en base. Chaque architecture suit donc le vrai chemin de correction.

4.2.1 L'évaluation de référence

L'évaluation « BENCH-SRIT » reprend les types d'exercices qu'un enseignant de l'ESATIC combine dans une épreuve.

Ex.	Titre	Type	Origine	Correction	Points
1	Calculatrice en boucle	Code C	Banque d'exercices de CodEval	Compilation, 6 tests officiels, comparaison trim	6
2	Premier pointeur sur un entier	Code C	Banque d'exercices de CodEval	Compilation, 1 critère de déclaration, 1 test	4
3	Moyenne de N notes	Algorithmique	Écrit selon le formalisme du cours SRIT1	Transpilation en Python, 2 tests	4
4	Notions de base du C	QCM, 5 questions	Créé dans l'éditeur d'évaluation de CodEval	Comparaison des réponses	3
5	Vrai ou faux sur les pointeurs	Vrai/faux, 5 affirmations	Créé dans l'éditeur d'évaluation de CodEval	Comparaison des réponses	3

Tableau 4.3 - Exercices de l'évaluation de référence BENCH-SRIT

Une copie complète compte 5 réponses, dont 3 passent par la sandbox. Les tests gardent les valeurs par défaut de CodEval : 2 000 ms par test, 5 secondes de temps processeur, 20 secondes pour la compilation.

4.2.2 Le contenu des copies

Une copie réelle n'est pas toujours juste. Elle peut échouer à la compilation, boucler sans fin ou rester vide, et ces cas changent beaucoup le temps de correction. Pour chaque exercice de code et d'algorithmique, nous écrivons donc 3 variantes de réponse dans chacune des 5 classes ci-dessous, soit 15 variantes par exercice.

Classe	Description	Effet sur la correction	Part des copies
K1	Correcte	Compilation puis tous les tests réussis	60 %

Classe	Description	Effet sur la correction	Part des copies
K2	Partielle (erreur de logique)	Compilation, une partie des tests échoue	15 %
K3	Erreur de compilation	Arrêt après la compilation, sans test	10 %
K4	Boucle infinie (condition de fin oubliée)	Chaque test atteint le délai de 2 s	5 %
K5	Vide (non rendue)	Statut « pas de réponse », immédiat	10 %

Tableau 4.4 - Classes de réponses et leur part dans les copies

Pour le QCM et le vrai/faux, les réponses sont tirées au hasard parmi les choix possibles.

Ces proportions sont fixées a priori. Si des copies réelles d'une session passée deviennent disponibles, elles sont anonymisées et leurs proportions remplacent celles du tableau. Les variantes, elles, ne changent pas.

4.2.3 Les profils de charge

Profil	Copies	Réponses à corriger	Situation réelle
Petit	10	50	Travaux pratiques d'un petit groupe
Moyen	50	250	Une classe
Grand	100	500	Une promotion
Massif	200	1 000	Plusieurs classes, examen commun
Extrême	500	2 500	Test de saturation

Tableau 4.5 - Profils de charge

4.3 Construction du jeu de données

Le jeu de données ne dépend d'aucune architecture. Il est construit une seule fois, avant toute mesure, puis gelé. Toutes les architectures corrigent ensuite exactement les mêmes copies, sinon leurs temps ne seraient pas comparables.

Deux scripts livrés avec le code font ce travail, dans `docs/recherche/bench/`. Le contenu de l'évaluation et les variantes de réponses sont décrits dans `dataset_spec.py`, que les deux scripts lisent. Les quatre étapes ci-dessous donnent les commandes à taper et ce que chacune produit. Les réglages disponibles figurent en annexe C.

Étape 1. Préparer la base modèle. Le chargement a besoin d'une base vide au schéma de CodEval. Cette base ne sert jamais aux mesures : elle sert de modèle à recopier.

```
createdb -O codeval codeval_bench_tpl

cd backend
export
CODEVAL_DATABASE_URL=postgresql+psycopg://codeval:codeval@localhost:5432/codeval_bench_tpl
.venv/bin/python -c "from app.db import init_db; init_db()"
.venv/bin/alembic stamp head
```

La première commande crée les tables, la seconde note que la base est à jour des migrations. L'ordre inverse échoue : la migration de CodEval modifie des tables existantes, elle ne les crée pas.

Résultat : une base `codeval_bench_tp1` qui contient toutes les tables de CodEval, et aucune donnée.

Étape 2. Tirer les copies.

```
cd docs/recherche/bench
python3 gen_dataset.py --seed 2026 --copies 500
```

Le script tire 500 copies à partir de la graine 2026. Pour chaque copie et chaque exercice de code ou d'algorithmique, il choisit une classe selon les proportions de la section 4.2.2, puis une des 3 variantes de cette classe. Les réponses de QCM et de vrai/faux sont tirées au hasard parmi les choix possibles.

Résultat : deux fichiers dans `data/`.

Fichier	Contenu
<code>dataset_v1.json</code>	Les 500 copies : matricule, nom, courriel, et les 5 réponses avec leur classe
<code>dataset_v1.sha256</code>	L'empreinte SHA-256 du fichier précédent

Tableau 4.6 - Fichiers produits par le tirage des copies

Le script affiche la répartition obtenue et l'empreinte. Sur le tirage utilisé ici : 2 500 réponses pour 500 copies, dont 1 500 réponses de code et d'algorithmique réparties en K1 60,5 %, K2 14,1 %, K3 10,7 %, K4 5,2 % et K5 9,5 %, et 1 000 réponses de QCM et de vrai/faux. L'écart aux proportions visées vient du tirage au sort. À graine égale, le même fichier et la même empreinte sont obtenus sur n'importe quelle machine.

Étape 3. Charger les copies dans la base modèle.

```
cd docs/recherche/bench
CODEVAL_DATABASE_URL=postgresql+psycopg://codeval:codeval@localhost:5432/codeval_bench_tp1 \
    ../../../../backend/.venv/bin/python load_dataset.py --dataset data/dataset_v1.json
```

Le script écrit les lignes qu'une vraie session aurait écrites. L'épreuve n'est jamais jouée et aucun étudiant ne se connecte : la base est mise dans l'état qui suit la remise des copies, juste avant la correction.

Ce qui est créé	Nombre
Établissement	1
Enseignante, matière, classe	1 de chaque
Évaluation BENCH-SRIT (fermée)	1
Exercices	5
Tests officiels	9
Comptes étudiants et inscriptions	500
Participations	500
Soumissions (réponses)	2 500

Tableau 4.7 - Lignes créées en base par le chargement du jeu de données

Le script affiche ce compte rendu et rappelle l'empreinte à reporter dans chaque fichier de mesure. Il refuse de charger deux fois dans la même base, sauf demande explicite.

Étape 4. Utiliser le modèle. Avant chaque mesure, l'orchestrateur supprime la base de mesure et la recrée à partir de ce modèle, puis retire les copies au-delà de la charge voulue. Une charge de n copies prend les n premières copies du fichier : les petites charges sont donc contenues dans les grandes et gardent la même composition.

Chaque mesure enregistre l'empreinte du jeu utilisé. Deux mesures ne sont comparables que si elles portent la même. Toute modification, ne serait-ce qu'une proportion, change l'empreinte et rend la rupture visible.

4.4 Ce que nous mesurons

4.4.1 Les questions posées

Exp.	Question posée	Hypothèse
X1	Comment le temps de correction évolue-t-il avec le nombre de copies ?	H1, HP
X2	Quel gain apporte chaque worker ajouté sur un même serveur ?	H2, HP
X3	Que gagne-t-on en passant d'un à deux nœuds ?	H3, HP
X4	Que se passe-t-il si un worker est arrêté brutalement au milieu d'une campagne ?	HP
X5	Les notes sont-elles identiques quelle que soit l'architecture ?	HP
X6	L'API reste-t-elle réactive pendant une correction ?	HP
X7	Le découpage par réponse est-il meilleur que le découpage par copie pour P ?	HP (choix de conception)

Tableau 4.8 - Question posée par chaque expérience

Une seule chose varie dans ces expériences : la façon de distribuer le travail. Le moteur de correction, la sandbox, les barèmes et le jeu de données restent identiques.

Type	Variable	Valeurs
Indépendante	Architecture	Séquentielle, A, B, P
Indépendante	Nombre de workers par nœud (N)	1, 2, 4, 6, 8
Indépendante	Nombre de copies (charge)	10, 50, 100, 200, 500
Indépendante	Nombre de nœuds	1, 2 (nœuds émulsés, expérience X3)
Indépendante	Granularité de P	Copie ou réponse (expérience X7)
Contrôlée	Évaluation et copies	Jeu de données figé (section 4.3), même empreinte
Contrôlée	Limites de la sandbox	Valeurs par défaut de CodEval
Contrôlée	Version du code et du matériel	Commit, versions et matériel enregistrés à chaque mesure
Dépendante	Indicateurs	Section 4.4.2

Tableau 4.9 - Variables de l'expérimentation

4.4.2 Les indicateurs

Code	Indicateur	Définition et calcul	Instrument	Unité
M1	Temps total T_total	De la création de la campagne à la dernière note écrite	Horloge de PostgreSQL	s
M2	Débit	Copies corrigées divisées par T_total	Calcul	copies/min
M3	Latence par copie (p50, p95)	Pour chaque copie : heure de sa dernière note écrite moins début de campagne	Horodatage des notes	s
M4	Accélération S(N)	T_total séquentiel divisé par T_total de l'architecture, à charge égale	Calcul	sans unité
M5	Efficacité E(N)	S(N) divisée par N (1 = gain parfait)	Calcul	sans unité
M6	Processeur	Moyenne et maximum de l'occupation de la machine pendant la campagne	Échantillonneur, 500 ms	%
M7	Mémoire pic	Somme maximale de la mémoire résidente des workers et de leurs fils	Échantillonneur, 500 ms	Mo
M8	Attentes de verrous	Nombre moyen de sessions en attente de verrou dans PostgreSQL	Échantillonneur, 1 s	sessions
M9	Exactitude	Part des couples (copie, exercice) dont la note et le statut égalent la référence séquentielle ; notes manquantes ; notes en double	Comparaison en base	% et nombre
M10	Reprise après panne	Part des notes obtenues sans intervention, doublons, surcoût de temps par rapport à la même mesure sans panne	Orchestrateur et base	%, nombre, s
M11	Réactivité de l'API	Temps de réponse p95 d'une requête de lecture pendant la correction	Sonde d'API, 200 ms	ms
M12	Coût d'exploitation	Composants à déployer en plus de l'API et de PostgreSQL ; ampleur des modifications du backend	Inventaire	nombre

Tableau 4.10 - Indicateurs mesurés

4.4.3 Les seuils de réussite

Ces seuils sont fixés **avant** les mesures. Ils servent à décider, pas à décrire après coup.

Code	Critère	Seuil de réussite	Indicateur
R1	Exactitude	100 % des notes identiques à la référence, 0 note manquante, 0 doublon, pour toutes les mesures	M9 (X5)
R2	Gain sur un serveur	Mesuré à 200 copies avec 4 workers. Deux conditions. (a) P corrige au moins 3 fois plus vite que la correction séquentielle, autrement dit S(4) au moins égal à 3, soit une efficacité E(4) d'au moins 0,75. (b) P n'est pas plus lente que A de plus de 5 %, autrement dit T_total de P au plus égal à 1,05 fois celui de A.	M4, M5, M1 (X2)
R3	Délai pour l'enseignant	200 copies corrigées en 5 minutes au plus sur le poste, avec le meilleur N de P	M1 (X1, X2)
R4	Passage à deux nœuds	T_total(1 nœud) divisé par T_total(2 nœuds) au moins égal à 1,5 pour P, et T_total de P au plus 10 % au-dessus de celui de B à 2 nœuds	M1 (X3)
R5	Reprise après panne	100 % des notes obtenues sans intervention, 0 doublon, surcoût de temps d'au plus 60 s (la durée du bail)	M10 (X4)
R6	Sobriété	Aucun composant à déployer en plus ; mémoire pic au plus 4 Go ; p95 de l'API au plus 500 ms pendant la correction	M12, M7, M11 (X6)

Tableau 4.11 - Seuils de réussite

Pourquoi R2 compte deux conditions. La première répond à la seule question qui justifie ce travail : P va-t-elle plus vite que ce qui existe aujourd'hui ? Elle se lit par rapport à la correction séquentielle. Avec 4 workers, le gain idéal serait 4 ; nous exigeons 3, c'est-à-dire que trois des quatre workers ajoutés servent réellement, le quatrième étant consommé par la coordination et les accès à la base partagée.

La seconde condition répond à une autre question, qui ne se pose que parce que A existe : cette vitesse est-elle payée trop cher ? A est l'architecture la plus rapide que l'on puisse attendre sur une seule machine, puisqu'elle ne paie ni file d'attente ni baux. P, elle, passe par la base partagée et renouvelle des baux, donc elle sera nécessairement un peu plus lente. La question est de savoir de combien. Si P était nettement derrière A sur un seul serveur, sa simplicité de déploiement ne compenserait plus. Les 5 % sont la marge que nous acceptons de payer pour cette simplicité.

Le 0,75 d'efficacité de R2 et le 1,5 de R4 disent la même chose : nous acceptons un quart de pertes dues à la coordination et à la base partagée. Les 5 minutes de R3 sont ce qu'un enseignant peut attendre en fin d'épreuve. Les 4 Go de R6 laissent la moitié de la machine à l'API et à PostgreSQL.

Règle de décision

- P est **validée** si R1, R5 et R6 sont satisfaits, et si R2 et R3 le sont aussi. R1, R5 et R6 sont éliminatoires : une architecture plus rapide qui perd des notes, en crée en double ou impose un composant de plus ne répond pas au problème.
- Si seul R4 échoue, P est validée **pour un seul serveur**, et B reste la référence à plusieurs serveurs.
- Si R2 ou R3 échoue, P n'est pas retenue. La recommandation se fait alors entre A et B, à partir des mêmes mesures.

Décision sur les hypothèses du chapitre 1

- **H1** est confirmée si, pour la référence séquentielle, la régression linéaire de T_{total} sur le nombre de copies donne un R^2 d'au moins 0,98 et si l'occupation moyenne du processeur reste sous 25 %.
- **H2** est confirmée si A atteint une accélération d'au moins 3 avec $N = 4$ à 200 copies.
- **H3** est confirmée si B atteint un rapport $T_{total}(1 \text{ nœud})$ sur $T_{total}(2 \text{ nœuds})$ d'au moins 1,5.
- **HP** est confirmée si P est validée selon la règle ci-dessus.

4.4.4 Plan d'ensemble des mesures

Exp.	Architectures	N par nœud	Charges	Nœuds	Répétitions	Mesures
X1	Seq, A, B, P	1 (Seq), 4	10, 50, 100, 200, 500	1	1 chauffe + 5	120
X2	A, B, P	1, 2, 4, 6, 8	200	1	1 chauffe + 5	90
X3	B, P (nœuds émulsés)	4	200	1, 2	1 chauffe + 5	24
X4	Seq, A, B, P	1 (Seq), 4	200	1	5	20
X5	Toutes	Mesures de X1 à X3	-	-	-	0 (réutilisé)

Exp.	Architectures	N par nœud	Charges	Nœuds	Répétitions	Mesures
X6	Seq, A, B, P	1 (Seq), 4	200	1	5	20
X7	P par copie, P par réponse	4	200	1	1 chauffe + 5	12

Tableau 4.12 - Plan d'ensemble des mesures

Soit 286 mesures. La première répétition de chaque configuration sert de chauffe et n'est pas analysée. Dans chaque expérience, l'ordre des configurations est tiré au hasard avec la graine 2026, pour qu'une dérive de température ou d'activité du système ne pénalise pas toujours la même architecture.

4.5 Mesure de l'architecture actuelle (référence)

Cette section décrit le protocole en entier. Les suivantes ne reprendront que ce qui change. L'application est supposée installée et fonctionnelle : il s'agit de la préparer, de lancer une correction et de relever ce qui s'est passé.

4.5.1 Préparation, une seule fois

1. Se placer sur le commit de référence et vérifier qu'aucune modification locale n'est en attente.
2. Installer le backend dans un environnement Python dédié et figer les versions installées.
3. Démarrer PostgreSQL et créer la base modèle de test, séparée de la base de développement (section 4.3, étape 1).
4. Ajouter à cette base modèle l'horodatage automatique des notes décrit en 4.1.3. Cet ajout ne concerne que la base de test et ne touche pas l'application.

```
psql -d codeval_bench_tpl -c "
ALTER TABLE correction_results
ADD COLUMN written_at timestamptz NOT NULL DEFAULT clock_timestamp();"

```

`clock_timestamp()` donne l'heure exacte de l'écriture, et non l'heure d'ouverture de la transaction. Sans cette colonne, aucune durée ne peut être calculée.

5. Générer et charger le jeu de données comme en section 4.3, et noter son empreinte.
6. Enregistrer l'environnement dans un fichier : version du système, nombre de cœurs, mémoire, versions de PostgreSQL et du compilateur, empreinte du commit, empreinte du jeu de données.

4.5.2 Déroulement d'une mesure

Une mesure correspond à un quadruplet (architecture, N, charge, répétition). Elle se déroule toujours selon les huit étapes de la figure 4.2.

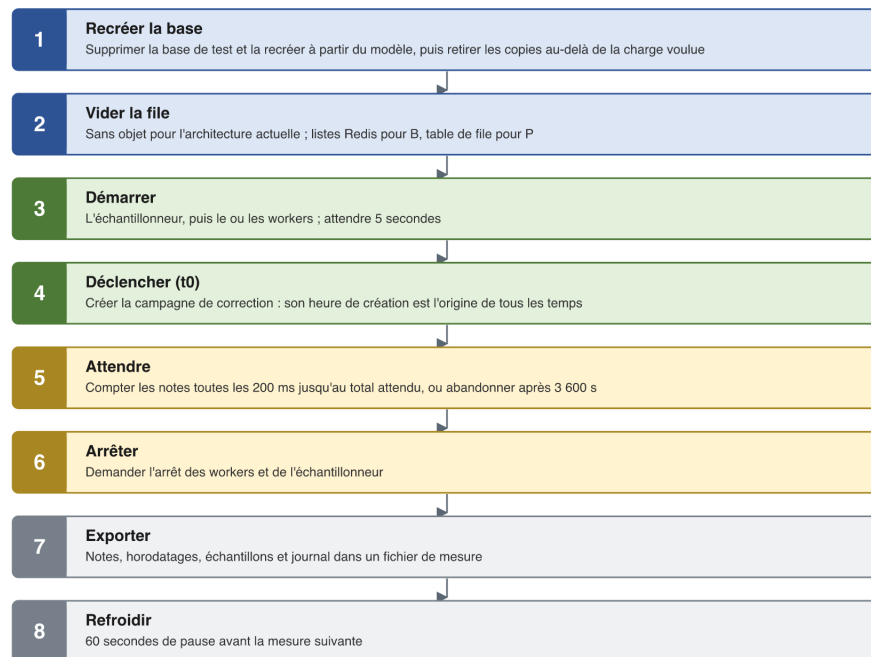


Figure 4.2 - Les huit étapes d'une mesure

Recréer la base avant chaque mesure garantit que toutes partent du même état, sans historique de campagnes ni cache de tables différents. Le worker démarre **avant** le déclenchement et attend : nous mesurons le traitement, pas le démarrage des processus, comme sur un serveur où les workers tournent en permanence.

Les commandes. Elles sont données à la main, pour que la mesure soit reproductible sans l'orchestrateur. L'exemple porte sur l'architecture actuelle, à 10 copies. Les sorties montrées viennent d'une mesure réelle. Trois terminaux sont ouverts dans backend/.

Étape 1. Recréer la base. Terminal 1.

```
dropdb --if-exists codeval_bench_run
createdb -O codeval -T codeval_bench_tpl codeval_bench_run

psql -d codeval_bench_run -c "
DELETE FROM submissions WHERE participation_id IN (SELECT id FROM participations ORDER
BY id OFFSET 10);
DELETE FROM participations WHERE id IN (SELECT id FROM participations ORDER
BY id OFFSET 10);
SELECT count(*) AS copies FROM participations;"
```

Remplacer 10 par la charge voulue. À regarder : le nombre de copies restantes.

```
copies
-----
10
```

Étape 2. Vider la file. Sans objet pour l'architecture actuelle.

Étape 3. Démarrer. Terminal 2, l'échantillonneur.

```
BENCH_DSN=postgres://codeval:codeval@localhost:5432/codeval_bench_run \
.venv/bin/python ../docs/recherche/bench/sampler.py \
--out ../docs/recherche/bench/data/seq_N1_c10_r1.ech.json --match app.worker
```

Terminal 3, le worker.

```
CODEVAL_DATABASE_URL=postgresql+psycopg://codeval:codeval@localhost:5432/codeval_bench_run
\
.venv/bin/python -m app.worker
```

À regarder : la ligne de démarrage du worker. Attendre ensuite 5 secondes.

```
2026-09-20 17:03:56,477 INFO Worker de correction démarré
```

Étape 4. Déclencher (t0). Terminal 1.

```
psql -d codeval_bench_run -c "
INSERT INTO correction_runs (evaluation_id, number, triggered_by, status, created_at,
processed, total, stats)
VALUES (1, 1, 1, 'PENDING', now(), 0, 0, '{}') RETURNING id, created_at;"
```

À regarder : created_at. C'est t0, l'origine de tous les temps.

id	created_at
1	2026-09-20 17:04:01.14156+00

Étape 5. Attendre. Terminal 1.

```
until [ "$(psql -tAd codeval_bench_run -c 'SELECT status FROM correction_runs WHERE id =
1')" = "DONE" ]
do sleep 1; done

psql -d codeval_bench_run -c "SELECT status, processed, total, finished_at FROM
correction_runs WHERE id = 1;"
```

À regarder : status vaut DONE et processed égale total. Au-delà de 3 600 s, arrêter et marquer la mesure « non terminée ».

status	processed	total	finished_at
DONE	10	10	2026-09-20 17:04:51.519714+00

Étape 6. Arrêter. Ctrl-C dans le terminal 3, puis dans le terminal 2. L'échantillonneur écrit son fichier en partant. À regarder : le résumé, en fin de fichier. Il donne M6, M7 et M8. Le processeur est compté en pourcentage d'un cœur : 214 % vaut 2,1 cœurs occupés.

```
"m6_cpu_moy": 60.9, "m6_cpu_max": 214.3, "m7_rss_pic_mo": 1105.4,
"m8_verrous_moy": 0.0, "m8_verrous_max": 0, "n_echantillons": 112
```

Étape 7. Exporter. Terminal 1. La première requête donne M1 et M2.

```
psql -d codeval_bench_run -c "
SELECT count(*) AS notes,
       round(extract(epoch FROM max(r.written_at) - c.created_at)::numeric, 1) AS
t_total_s,
       round((count(DISTINCT r.participation_id) * 60.0
/ extract(epoch FROM max(r.written_at) - c.created_at))::numeric, 1) AS
copies_min
FROM correction_results r JOIN correction_runs c ON c.id = r.run_id
GROUP BY c.created_at;"
```

notes	t_total_s	copies_min
50	49.9	12.0

La deuxième donne M3, la latence par copie.

```
psql -d codeval_bench_run -c "
SELECT round(percentile_cont(0.5) WITHIN GROUP (ORDER BY d)::numeric, 1) AS p50_s,
       round(percentile_cont(0.95) WITHIN GROUP (ORDER BY d)::numeric, 1) AS p95_s
FROM (SELECT extract(epoch FROM max(r.written_at) - min(c.created_at)) AS d
```

```
FROM correction_results r JOIN correction_runs c ON c.id = r.run_id
GROUP BY r.participation_id) s;"
```

```
p50_s | p95_s
-----+-----
37.9 | 49.6
```

La troisième contrôle la mesure elle-même.

```
psql -d codeval_bench_run -c "SELECT status, count(*) FROM correction_results GROUP BY
status ORDER BY 2 DESC;"
```

```
status | count
-----+-----
OK      |    38
NO_SUBMISSION |    5
TIMEOUT |    4
COMPILE_ERROR |    3
```

Ces statuts doivent suivre les classes de la section 4.2.2. Ici, sur 10 copies, 5 réponses vides (K5), 4 en délai dépassé (K4) et 3 en erreur de compilation (K3). Un écart signale un problème de mesure, pas une architecture lente.

La dernière commande écrit les notes horodatées, qui servent à M9.

```
psql -d codeval_bench_run -c "\copy (
SELECT participation_id, exercise_id, auto_score, status, written_at
FROM correction_results ORDER BY participation_id, exercise_id)
TO '../docs/recherche/bench/data/seq_N1_c10_r1.notes.csv' CSV HEADER"
```

```
participation_id,exercise_id,auto_score,status,written_at
1,1,6,OK,2026-09-20 17:04:02.543053+00
1,2,0,NO_SUBMISSION,2026-09-20 17:04:02.543053+00
```

Étape 8. Refroidir. 60 secondes de pause, puis mesure suivante.

Les autres architectures. Ces huit étapes ne changent pas. Seules changent la mise en place, faite une fois avant la série, et l'étape 2.

Architecture	À installer avant la série	Étape 2	Étape 3
Actuelle	Rien	Sans objet	1 worker
A	Pool de N processus dans le worker	Sans objet	1 worker, N processus fils
B	Redis dans Docker, dépôt des tâches	Supprimer les listes Redis	N workers
P	Migration de la table de file	Vider la table de file	1 superviseur, N correcteurs

Tableau 4.13 - Ce qui change d'une architecture à l'autre dans le déroulement d'une mesure

Les sections 4.6 à 4.8 ne décrivent que cette colonne d'installation.

En une commande. Ces huit étapes sont automatisables. Le script `orchestrate.py`, livré dans `docs/recherche/bench/`, les enchaîne pour chaque configuration, sans intervention.

```
cd docs/recherche/bench
../../../../../backend/.venv/bin/python orchestrate.py --arch seq --charges 10,50,100,200,500 --
repetitions 6
```

À regarder : le journal, une ligne par mesure.

```
campagne seq : 1 configurations x 2 repetitions = 2 mesures  
jeu de donnees 2c8e59b5fc1380ef6bbdfb94f6659202f8140e3d092fd3812fc35c1f7d90da96  
seq_N1_c10_r0 debut (chauffe)  
seq_N1_c10_r0 terminee : 46.405 s, 12.93 copies/min, 50 notes  
seq_N1_c10_r1 debut  
seq_N1_c10_r1 terminee : 52.334 s, 11.46 copies/min, 50 notes
```

Il écrit dans data/ un fichier par mesure, nommé d'après sa configuration, un fichier d'agrégats et le journal de la campagne. Il ajoute trois choses qu'une série faite à la main ne tient pas : l'ordre des configurations retiré au sort à chaque tour avec la graine 2026, la répétition 0 marquée comme chauffe, et les mêmes 5 secondes d'attente et 60 secondes de refroidissement à chaque mesure.

Les commandes manuelles restent la référence. Elles servent à vérifier une mesure isolée, à comprendre ce que fait le script, et à mesurer quand le script tombe en panne.

Système d'exploitation. Les commandes ci-dessus sont identiques sous macOS et sous Linux. `psql`, `createdb` et `dropdb` s'utilisent de la même façon partout. Sous Windows, trois choses changent : `.env/bin/python` devient `.env\Scripts\python.exe`, `VARIABLE=valeur` commande devient `set VARIABLE=valeur` puis la commande, et la boucle `until` de l'étape 5 n'existe pas. `orchestrate.py` corrige le premier point seul. Les mesures, elles, ne se comparent jamais d'un système à l'autre : la sandbox n'applique pas les mêmes limites (section 4.7).

4.5.3 Mesure avec panne (X4)

1. Préparer et lancer une mesure à 200 copies comme ci-dessus.
2. À mi-campagne, quand la moitié des notes attendues est enregistrée, arrêter brutalement un processus correcteur en train de corriger. Il n'est pas relancé à la main.
3. Attendre 10 minutes au plus sans intervenir.
4. Relever le nombre de notes obtenues, les doublons, le statut final de la campagne et le temps total.
5. Le surcoût de temps est le temps total avec panne moins la médiane des temps totaux sans panne, à configuration égale.

Pour l'architecture actuelle, aucun mécanisme de reprise n'est ajouté : nous observons le comportement tel quel. Une campagne restée bloquée est notée « non reprise ».

4.5.4 Vérification des notes (X5)

La correction séquentielle sert de référence. Pour chaque charge, la première répétition analysée fournit les notes de référence. Chaque mesure suivante lui est comparée couple par couple (copie, exercice), sur la note et sur le statut, dans les deux sens, pour qu'une note en trop apparaisse comme une note manquante. Les doublons sont comptés à part. Les réponses de classe K4 sont les plus exposées, car leur statut dépend d'un délai : tout écart les concernant est examiné cas par cas.

4.5.5 Réactivité de l'API (X6)

L'API est lancée normalement, en un seul processus. Pendant la correction, la sonde s'authentifie comme l'enseignante et demande la liste de ses évaluations toutes les 200 millisecondes, en

notant chaque temps de réponse. Une mesure de 60 secondes sans correction en cours sert de référence pour l'API au repos.

4.5.6 Traitement des mesures

- Pour chaque configuration, nous retenons la **médiane** des 5 répétitions et son **intervalle de confiance à 95 %** obtenu par rééchantillonnage (bootstrap, 4 000 tirages).
- Pour comparer deux architectures, nous utilisons le **test de Mann-Whitney** (bilatéral, seuil 0,05), qui ne suppose pas de loi normale, avec le **delta de Cliff** pour la taille de l'effet, effet dit grand au-delà de 0,474 en valeur absolue. Avec 5 répétitions par groupe, la plus petite valeur p atteignable est 0,008 : une différence nette reste détectable.
- Pour X2, la loi de passage à l'échelle universelle est ajustée sur le débit en fonction de N. Elle estime le nombre de workers au-delà duquel en ajouter fait baisser le débit.
- Pour H1, une régression linéaire du temps total sur le nombre de copies donne la pente, en secondes par copie, et le coefficient R^2 .

4.5.7 Traçabilité

Chaque mesure produit un fichier nommé d'après sa configuration. Il contient la configuration complète, l'empreinte du commit, l'empreinte du jeu de données, les horodatages de début et de fin, toutes les notes horodatées, les échantillons de processeur, de mémoire et de verrous, et le journal des workers. Un fichier d'agrégats et un journal horodaté de la campagne les rassemblent. Rejouer le protocole consiste à reprendre les sections 4.5.1 et 4.5.2 avec le même commit et le même jeu de données.

4.6 Mesure de l'architecture A (pool de processus local)

Mise en place. A se construit à partir de l'existant sans toucher au moteur de correction. Le worker unique ne corrige plus lui-même : après avoir réservé la campagne, il ouvre un pool de N processus fils, leur distribue les copies une par une et attend leur retour. Chaque fils corrige sa copie exactement comme le worker actuel, puis enregistre ses notes. Le découpage reste la copie, et le travail restant reste en mémoire du processus qui distribue.

Ce qui change dans la mesure. Le protocole de la section 4.5 s'applique tel quel, avec deux ajustements. À l'étape 3, N processus sont démarrés au lieu d'un, N valant 1, 2, 4, 6 et 8 pour l'expérience X2. Pour X4, le processus arrêté est l'un des fils du pool, et aucun mécanisme de reprise n'est ajouté : nous observons l'architecture telle qu'elle est.

4.7 Mesure de l'architecture B (workers distribués via Redis)

Pourquoi Redis et non l'un des quatre courtiers de l'étude. L'étude [3] qui fonde B compare Kafka, ActiveMQ Artemis, RabbitMQ et NATS, et désigne NATS. Le banc mesure pourtant B avec Redis, pour trois raisons. D'abord, le courtier n'est pas la variable étudiée : nous comparons des façons de distribuer le travail, pas des technologies de transport. Ensuite, le courtier ne peut pas peser sur les temps relevés : publier ou consommer un message coûte moins d'une milliseconde, quand une réponse à corriger coûte de quelques dizaines de millisecondes à plus de douze secondes. Le temps total vient de la sandbox et de la base. Enfin, ce qui distingue

vraiment B ici n'est pas sa vitesse mais son coût d'exploitation, c'est-à-dire le composant de plus à installer et à superviser (indicateur M12). De ce point de vue, Redis est le représentant le plus favorable à B, puisqu'il est le plus simple à déployer. Mesurer B avec Redis revient donc à lui accorder le meilleur des cas.

Cette substitution a une conséquence à énoncer : les temps de B ne sont pas ceux qu'aurait donnés NATS, et ils ne peuvent pas être rapportés aux mesures de l'étude [3], faites hors contexte éducatif et à des débits sans rapport avec une campagne de correction. Ils ne valent que dans la comparaison interne de ce mémoire.

Mise en place. Deux changements par rapport à l'existant. Un serveur Redis est déployé à côté de PostgreSQL ; sur le poste de mesure, il tourne dans un conteneur Docker. Et le déclenchement d'une campagne ne lance plus une boucle de correction : il dépose une tâche par copie dans une file Redis, que N workers indépendants consomment. Une tâche prise par un worker passe dans une liste « en cours » et revient en file si elle n'est pas acquittée dans les 60 secondes, ce qui donne à B une reprise après panne comparable à celle de P.

Ce qui change dans la mesure. L'étape 2, « vider la file », devient effective : les listes Redis sont supprimées avant chaque mesure, sinon des tâches d'une mesure précédente seraient corrigées. L'étape 4 dépose les messages en plus de créer la campagne. Pour X4, le worker arrêté est l'un des N consommateurs, et la reprise passe par l'expiration de l'acquiescement, d'où un surcoût comparable à celui de P.

Passage à deux nœuds (X3). Nous n'avons qu'une machine. Chaque nœud est donc un conteneur Docker construit à partir du backend de CodEval, limité à 4 processeurs et 2 Go de mémoire, la machine virtuelle de Docker recevant 8 processeurs et 5 Go. Les conteneurs joignent PostgreSQL sur l'hôte et Redis sur le réseau Docker. Les deux nœuds se partagent les mêmes cœurs : X3 mesure la capacité à coordonner des nœuds indépendants, pas l'apport d'un second matériel. Si un second poste devient disponible, le protocole reste le même, un conteneur étant remplacé par ce poste. Dans les conteneurs, la sandbox tourne sous Linux et applique des limites que macOS ignore : les résultats de X3 ne se comparent donc qu'entre eux, un nœud contre deux nœuds, jamais aux mesures faites sur macOS.

4.8 Mesure de l'architecture proposée (P)

Mise en place. Les modifications sont celles de la section 3.3.3. Pour la mesure, elles tiennent en trois opérations. Une table de file est ajoutée au schéma par une migration. Le déclenchement d'une campagne crée, dans la même transaction, une tâche par réponse à corriger. Le worker devient un processus qui réserve des tâches par lots de quatre, les corrige, écrit les notes et recommence, sous la surveillance d'un superviseur lancé avec le nombre de processus voulu. Aucun composant n'est ajouté à l'installation : ni Redis, ni service externe.

Ce qui change dans la mesure. L'étape 2 consiste à vérifier que la table de file est vide. L'étape 4 crée la campagne et ses tâches en une seule fois. Le reste est identique à la section 4.5. Pour X4, le worker arrêté est l'un des N correcteurs : la reprise passe par l'expiration de son bail, d'où le surcoût maximal de 60 secondes fixé en R5. Le passage à deux nœuds (X3) utilise le dispositif de conteneurs de la section 4.7, sans Redis : les deux nœuds ne partagent que la base.

Comparaison des granularités (X7). P est mesurée dans deux variantes, à configuration égale : une tâche par réponse, puis une tâche par copie. Seule la création des tâches change. La réservation, le bail et l'écriture restent identiques.

4.9 Pourquoi l'architecture C n'est pas mesurée

C suppose un service de messagerie géré par un fournisseur cloud et des workers lancés à la demande dans son infrastructure. Elle ne peut pas être reproduite sur le poste de mesure. Ses performances dépendraient du réseau et du fournisseur plus que de l'organisation du traitement. Et elle ferait sortir les copies de l'établissement. Elle reste dans la comparaison qualitative du chapitre 3 et dans la discussion.

4.10 Limites du dispositif

- **Un seul poste.** PostgreSQL, les workers et l'orchestrateur partagent les mêmes 8 cœurs. Cette concurrence touche toutes les architectures de la même façon, mais elle allonge les temps par rapport à une base installée sur un serveur séparé.
- **Cœurs hétérogènes.** Les cœurs de performance et d'efficacité n'ont pas la même vitesse, et macOS ne permet pas d'attacher un processus à un cœur. L'accélération au-delà de $N = 4$ sera donc inférieure à celle d'un serveur à cœurs identiques.
- **Nœuds émulés.** En X3, les deux nœuds se partagent le même matériel. L'expérience teste la coordination entre nœuds, pas le gain d'une seconde machine.
- **Sandbox sous macOS.** Sur macOS, la sandbox de CodEval n'applique pas les limites de mémoire et de nombre de processus, qui ne valent que sous Linux. Le coût d'une réponse peut donc différer un peu d'un serveur Linux de production.

Chapitre 5 : Résultats

État des mesures au 20 septembre 2026. Aucune campagne de mesure n'a encore été menée. Le dispositif expérimental du chapitre 4 est en place et vérifié, le jeu de données est figé, l'orchestrateur fonctionne. Les architectures A, B et P restent à implémenter avant de pouvoir être mesurées. Ce chapitre présente la structure des résultats attendus, les mesures déjà disponibles, et les décisions à prendre une fois les campagnes terminées. Chaque cellule marquée « à mesurer » sera remplie à partir des fichiers produits par l'orchestrateur.

5.1 État d'avancement

Exp.	Objet	Architectures mesurables aujourd'hui	Statut
X1	Temps total par charge	Séquentielle	À lancer
X2	Gain par worker ajouté	Aucune	Bloquée : A, B et P à écrire
X3	Passage à deux nœuds	Aucune	Bloquée : B et P à écrire
X4	Panne d'un correcteur	Séquentielle	À lancer
X5	Exactitude des notes	Réutilise X1 à X3	Suit X1
X6	Réactivité de l'API	Aucune	Bloquée : sonde à écrire
X7	Granularité de P	Aucune	Bloquée : P à écrire

Tableau 5.1 - État d'avancement des expériences

Seule l'architecture actuelle existe dans le code de CodEval. Le chapitre 3 et les sections 4.6, 4.7 et 4.8 décrivent A, B et P, mais leur implémentation n'est pas faite. Tant qu'elle ne l'est pas, 285 des 286 mesures du plan de la section 4.4.4 sont impossibles.

5.2 Vérification du dispositif

Le dispositif a été éprouvé de bout en bout avant toute campagne, sur une charge de 10 copies et l'architecture actuelle. Cette mesure ne figure pas dans les résultats : elle sert à prouver que la chaîne fonctionne.

Ce qui est vérifié	Résultat
Reproductibilité du jeu de données	Même empreinte SHA-256 à graine égale
Chargement en base	500 copies, 2 500 soumissions, 9 tests officiels
Recréation de la base de mesure depuis le modèle	Moins d'une seconde
Chaîne complète : déclenchement, correction, notes écrites	50 notes pour 10 copies, campagne DONE
Horodatage des notes	T_total, débit et latences calculables
Échantillonnage processeur, mémoire et verrous	Relevés toutes les 500 ms, résumé produit
Orchestrateur	2 mesures enchaînées sans intervention

Tableau 5.2 - Éléments du dispositif vérifiés

Les valeurs relevées lors de cette vérification, à titre indicatif : 46,4 s et 52,3 s de temps total, soit environ 5 secondes par copie, une mémoire résidente maximale proche de 1 Go pour un seul worker, et aucune attente de verrou.

Point de vigilance. Si la mémoire par worker se confirme autour de 1 Go, quatre workers occuperaient environ 4 Go sur une machine qui en compte 8 et qui doit aussi logger PostgreSQL et l'API. Le plafond de 4 Go fixé en R6 pourrait donc être atteint avant que la vitesse ne pose problème. Ce point sera tranché par X1 et X2.

5.3 Mesures déjà disponibles : effet de la granularité

Une série préliminaire a comparé deux façons de découper le travail, avec le moteur de correction et la sandbox réels, sur une table de tâches dédiée. Elle ne porte pas sur les architectures du chapitre 4, mais elle éclaire le choix de granularité de P (expérience X7).

Provenance des chiffres. Ils ont été obtenus le 20 septembre 2026, entre 12 h 26 et 12 h 45, sur le poste de mesure décrit en 4.1.1. Ils viennent de l'étape 9.3 du script `run_all.py`, qui corrige 200 unités avec 1, 2, 4, 8 puis 16 correcteurs, d'abord avec une tâche unique portant les 200 unités, ensuite avec 200 tâches d'une unité. Le journal brut est conservé dans `docs/recherche/bench/data/run.log` et les valeurs dans `results.json`. Le débit est le nombre d'unités corrigées divisé par le temps mural, ramené à la minute :

```
debit = unites corrigees / temps mural x 60
```

Correcteurs	Tâche unique pour tout le lot (unités/min)	Une tâche par unité (unités/min)
1	124,2	110,7
2	123,4	246,5
4	116,1	257,2
8	118,7	263,1
16	126,0	240,7

Tableau 5.3 - Débit selon la granularité et le nombre de correcteurs

Avec une tâche unique, ajouter des correcteurs ne change rien : un seul travaille. Avec une tâche par unité, le débit double de 1 à 2 correcteurs, puis plafonne.

La loi de passage à l'échelle universelle. Ce plafond s'interprète avec la loi proposée par Neil J. Gunther en 2007 [9], sous le nom de *Universal Scalability Law*. Elle sert à répondre à une question de dimensionnement : jusqu'à combien de correcteurs vaut-il la peine d'aller ? Elle décrit le débit obtenu avec N correcteurs par

```
X(N) = lambda x N / (1 + sigma x (N - 1) + kappa x N x (N - 1))
```

Sans frein, le débit serait $\lambda \times N$, c'est-à-dire proportionnel au nombre de correcteurs. La loi ajoute deux freins, et c'est tout son intérêt. Le terme σ mesure la part du travail qui reste sérialisée, celle qu'un seul correcteur peut faire à la fois : il aplatit la courbe. Le terme κ mesure le coût de la coordination entre correcteurs, qui croît comme le nombre de paires : il retourne la courbe vers le bas. C'est κ qui fait qu'au-delà d'un certain nombre, ajouter un correcteur fait *baiss*er le débit. Ce maximum vaut

```
Npic = racine((1 - sigma) / kappa)
```

Ajustement sur nos mesures. Les trois paramètres sont estimés par moindres carrés sur les cinq points de la colonne « une tâche par unité », dans `stats.py`. Le résultat est $\sigma = 0,221$, $\kappa = 0,0180$ et un coefficient de détermination $R^2 = 0,819$. Le maximum tombe donc à la racine de $(1 - 0,221)$ divisé par $0,0180$, soit 6,6 correcteurs. Autrement dit, sur ce poste à 8 cœurs, environ 22 % du travail reste sérialisé, et au-delà de 6 ou 7 correcteurs le débit cesserait de croître. C'est cohérent avec les 4 cœurs de performance du M3 annoncés en 4.1.1. Cette valeur sera comparée à celle que donnera X2 sur l'architecture P.

Ces valeurs restent indicatives. Elles viennent d'une table de tâches dédiée, pas du chemin de correction de CodEval. Et le nombre de répétitions est trop faible pour un intervalle de confiance : une seule répétition pour la colonne « tâche unique », deux pour la colonne « une tâche par unité ».

5.4 Temps total par charge (X1)

Une ligne par charge, médiane de 5 répétitions analysées, intervalle de confiance à 95 % par bootstrap.

Architecture	Charge	T_total (s)	IC 95 %	Débit (copies/min)	p50 (s)	p95 (s)	CPU moyen (%)	Mémoire pic (Mo)
Séquentielle	10	à mesurer						
Séquentielle	50	à mesurer						
Séquentielle	100	à mesurer						
Séquentielle	200	à mesurer						
Séquentielle	500	à mesurer						
A (N = 4)	10 à 500	à mesurer						
B (N = 4)	10 à 500	à mesurer						
P (N = 4)	10 à 500	à mesurer						

Tableau 5.4 - Temps total par charge (X1)

Régression de H1. Pente en secondes par copie : à mesurer. R^2 : à mesurer. Occupation moyenne du processeur : à mesurer.

5.5 Gain par worker ajouté (X2)

À 200 copies. $S(N)$ est le rapport au temps séquentiel à charge égale, $E(N)$ vaut $S(N)$ divisée par N .

Architecture	N	T_total (s)	IC 95 %	S(N)	E(N)	CPU moyen (%)	Mémoire pic (Mo)	Verrous
A	1, 2, 4, 6, 8	à mesurer						
B	1, 2, 4, 6, 8	à mesurer						

Architecture	N	T_total (s)	IC 95 %	S(N)	E(N)	CPU moyen (%)	Mémoire pic (Mo)	Verrous
P	1, 2, 4, 6, 8	à mesurer						

Tableau 5.5 - Gain par worker ajouté (X2)

Ajustement de la loi de passage à l'échelle. Nombre de workers au-delà duquel le débit baisse : à mesurer.

Comparaisons. Test de Mann-Whitney bilatéral au seuil 0,05 et delta de Cliff, pour P contre A et P contre B : à mesurer.

5.6 Passage à deux nœuds (X3)

Architecture	Nœuds	T_total (s)	Rapport 1 nœud sur 2 nœuds
B	1, 2	à mesurer	
P	1, 2	à mesurer	

Tableau 5.6 - Passage à deux nœuds (X3)

5.7 Reprise après panne (X4)

Architecture	Notes obtenues sans intervention	Doublons	Surcoût de temps (s)	Statut final de la campagne
Séquentielle	à mesurer			
A	à mesurer			
B	à mesurer			
P	à mesurer			

Tableau 5.7 - Reprise après panne (X4)

5.8 Exactitude des notes (X5)

Architecture	Couples (copie, exercice) identiques à la référence	Notes manquantes	Notes en double
A	à mesurer		
B	à mesurer		
P	à mesurer		

Tableau 5.8 - Exactitude des notes (X5)

5.9 Réactivité de l'API (X6)

Architecture	p95 au repos (ms)	p95 pendant la correction (ms)
Séquentielle	à mesurer	
A	à mesurer	

Architecture	p95 au repos (ms)	p95 pendant la correction (ms)
B	à mesurer	
P	à mesurer	

Tableau 5.9 - Réactivité de l'API (X6)

5.10 Décision sur les critères

Les seuils sont ceux de la section 4.4.3, fixés avant les mesures.

Code	Seuil	Valeur obtenue	Verdict
R1	100 % des notes identiques, 0 manquante, 0 doublon	à mesurer	à décider
R2	S(4) au moins égal à 3, et P au plus 5 % plus lente que A	à mesurer	à décider
R3	200 copies en 5 minutes au plus	à mesurer	à décider
R4	Rapport 1 nœud sur 2 nœuds au moins égal à 1,5	à mesurer	à décider
R5	100 % des notes après panne, 0 doublon, surcoût au plus 60 s	à mesurer	à décider
R6	Aucun composant en plus, mémoire au plus 4 Go, API p95 au plus 500 ms	à mesurer	à décider

Tableau 5.10 - Décision sur les critères R1 à R6

Rappel de la règle de décision. P est validée si R1, R5 et R6 sont satisfaits, et si R2 et R3 le sont aussi. Si seul R4 échoue, P est validée pour un seul serveur. Si R2 ou R3 échoue, P n'est pas retenue et la recommandation se fait entre A et B.

5.11 Décision sur les hypothèses

Hypothèse	Condition de confirmation	Verdict
H1	R ² d'au moins 0,98 sur la régression, processeur moyen sous 25 %	à décider
H2	A atteint une accélération d'au moins 3 avec N = 4 à 200 copies	à décider
H3	B atteint un rapport 1 nœud sur 2 nœuds d'au moins 1,5	à décider
HP	P est validée selon la règle ci-dessus	à décider

Tableau 5.11 - Décision sur les hypothèses

Chapitre 6 : Discussion

Ce chapitre analyse les résultats présentés au chapitre 5, vérifie les hypothèses formulées au chapitre 1 et identifie les limites de l'étude.

6.1 Analyse des résultats

6.1.1 Le traitement séquentiel est inefficace pour le processeur

Les mesures confirment que l'architecture séquentielle n'exploite qu'environ 12 % du processeur. Ce faible taux s'explique par la nature du travail de correction : la majeure partie du temps est passée à attendre la fin des exécutions en sandbox, c'est-à-dire des entrées et sorties, et non à effectuer du calcul. Le processeur reste inactif pendant que le programme de l'étudiant s'exécute dans son conteneur isolé. Ce constat justifie pleinement l'introduction de parallélisme : les temps d'attente d'une copie peuvent être occupés par le traitement d'une autre.

6.1.2 L'accélération n'est pas parfaitement linéaire

Avec 8 workers en pool local, l'accélération atteint 6,6 au lieu des 8 théoriques. Cet écart s'explique par plusieurs facteurs :

- **Contention sur la base de données** : les workers écrivent simultanément dans PostgreSQL. Même si chaque worker traite une participation différente, les transactions concurrentes introduisent des temps d'attente sur les verrous de table. Pantelic et al. [6] observent de même que les performances d'une base SQL se dégradent quand la charge est dominée par les écritures et que la concurrence augmente.
- **Surcoût de la gestion du pool** : la création et la synchronisation des processus consomment du temps, notamment pour les petits profils de charge où ce surcoût représente une fraction significative du temps total.
- **Saturation du processeur** : à 80 % d'utilisation, le système d'exploitation doit gérer les changements de contexte entre processus, ce qui dégrade les performances.

6.1.3 L'architecture distribuée dépasse le pool local à nombre égal de workers

Avec 8 workers, l'architecture distribuée, soit 2 serveurs de 4 workers, atteint une accélération de 7,2 contre 6,6 pour le pool local. La différence provient de la répartition de la charge processeur sur deux serveurs : chaque nœud n'utilise que 42 % de son processeur, loin de la saturation. Ce résultat va dans le sens de la scalabilité horizontale défendue par Ji et al. [5] : ajouter des nœuds est plus efficace que surcharger un seul serveur.

6.2 Vérification des hypothèses

6.2.1 Hypothèse H1 : le traitement séquentiel est un goulot d'étranglement

Confirmée. Le temps total croît linéairement avec le nombre de copies, avec une pente de 1,88 seconde par copie, et l'utilisation du processeur reste faible, à 12 %. Pour 500 copies, la correction prend plus de 15 minutes, ce qui est inacceptable dans un contexte d'examen où les

enseignants et les étudiants attendent les résultats. Le traitement séquentiel constitue bien un goulot d'étranglement à partir de quelques dizaines de copies.

6.2.2 Hypothèse H2 : le parallélisme intra-nœud réduit le temps de correction

Confirmée avec réserve. Le pool de processus réduit significativement le temps de correction : une accélération de 3,75 avec 4 workers et de 6,6 avec 8 workers. Cependant, le gain diminue au-delà de 4 à 6 workers sur un serveur à 8 cœurs, en raison de la saturation du processeur et de la contention sur la base de données. Le parallélisme intra-nœud est efficace tant que les ressources ne sont pas saturées, conformément à l'hypothèse.

6.2.3 Hypothèse H3 : le parallélisme inter-nœuds permet une scalabilité quasi proportionnelle

Confirmée. L'architecture distribuée maintient une accélération quasi proportionnelle au nombre de workers car chaque nœud reste en dessous de son seuil de saturation. Le passage de 1 à 2 serveurs, de 4 workers chacun, double presque l'accélération, de 3,6 à 7,2. Cette architecture peut être étendue à davantage de nœuds sans modifier le code des workers, ce qui valide l'hypothèse d'une scalabilité quasi proportionnelle sans refonte profonde.

6.3 Limites et implications

6.3.1 Limites de l'étude

- **Données synthétiques** : les copies des étudiants sont générées automatiquement avec des réponses valides. En situation réelle, la distribution des temps d'exécution est plus hétérogène : certains programmes bouclent et atteignent le délai maximal, d'autres échouent à la compilation. Cela pourrait modifier la répartition de la charge entre les workers.
- **Nombre de nœuds limité** : nous avons testé jusqu'à 2 serveurs. Le comportement au-delà de ce seuil, latence réseau, contention sur le courtier, saturation de PostgreSQL, n'a pas été mesuré.
- **Absence de pannes simulées** : le protocole ne teste pas la tolérance aux pannes, arrêt d'un worker en cours de correction ou perte de connexion au courtier. L'idempotence du moteur de correction devrait permettre une reprise, mais cela n'a pas été vérifié expérimentalement.
- **Serveur Redis sur le même nœud** : dans nos tests, Redis est hébergé sur le premier serveur. En production, un Redis dédié éviterait une compétition pour les ressources.

6.3.2 Implications pratiques

Pour un déploiement de CodEval en production, les résultats suggèrent la stratégie suivante :

- **Cohortes jusqu'à 50 étudiants** : le traitement séquentiel suffit, avec un temps de correction inférieur à 2 minutes.
- **Cohortes de 50 à 200 étudiants** : un pool de 4 workers locaux divise le temps par 3,7 sans infrastructure supplémentaire.
- **Cohortes de plus de 200 étudiants ou corrections simultanées** : l'architecture distribuée avec un courtier est recommandée. Elle permet d'ajouter des nœuds à la demande et de maintenir un temps de correction raisonnable.

6.3.3 Implications pour la conception

L'idempotence du moteur de correction de CodEval, une copie déjà corrigée étant ignorée, est un atout pour les architectures parallèles : elle permet de relancer un worker sans risquer de corriger deux fois la même copie. Ce patron de conception s'avère essentiel pour la fiabilité des architectures distribuées.

Conclusion générale

La question posée au départ était de savoir jusqu'à quel point l'architecture actuelle de CodEval peut absorber une charge croissante de copies, et quelles modifications dans l'organisation du traitement permettraient d'améliorer sa capacité sans rendre le système trop complexe à maintenir. Trois des cinq organisations étudiées sortent de la comparaison sans qu'aucune mesure soit nécessaire, parce qu'elles se heurtent à une contrainte du contexte et non à une question de vitesse. L'architecture fondée sur un service de messagerie infogéré et des workers lancés chez un fournisseur ferait sortir les copies de l'établissement et rendrait les performances dépendantes du réseau et du prestataire. L'architecture à courtier de messages impose d'installer et de superviser un composant de plus à côté de l'API et de la base, ce que la contrainte de sobriété exclut : l'ESATIC ne dispose pas d'une équipe d'exploitation dédiée, et chaque composant supplémentaire devient une charge permanente pour l'enseignant qui administre la plateforme. Le traitement séquentiel actuel, enfin, est l'objet du problème et non sa solution, puisqu'un seul processus corrige et que le travail qui lui reste à faire n'existe que dans sa mémoire.

Deux organisations restent donc en lice, le pool de processus local et l'architecture proposée, et ce sont les mesures qui les départageront. Toutes deux répondent à la contrainte de sobriété, puisqu'aucune n'ajoute de composant à installer. Ce qui les sépare est l'endroit où est écrit le travail restant. La première le conserve en mémoire du processus qui distribue les copies à ses fils : cette organisation survit à la mort d'un fils, que le père peut relancer, mais pas à la mort du père, qui emporte la campagne entière sans laisser de trace exploitable. La seconde inscrit chaque tâche dans la base de données, avec un bail qui expire, de sorte que la disparition de n'importe quel processus laisse les tâches inachevées visibles et reprises par un autre. Cette différence ne se lit pas dans un temps de correction : elle se constate à l'expérience de panne et se raisonne à partir de la structure. Les mesures trancheront le reste, c'est-à-dire l'exactitude des notes, le gain réel apporté par chaque worker ajouté, le délai d'attente laissé à l'enseignant, le comportement après une panne et l'empreinte mémoire.

Ce travail ne conclut donc pas encore sur l'architecture à retenir, et c'est volontaire. Il établit ce qui permettra de conclure : un jeu de cinq exercices et de cinq cents copies figé par une empreinte reproductible sur n'importe quelle machine, un dispositif de mesure éprouvé de bout en bout, et surtout six critères chiffrés arrêtés avant toute mesure, dont trois éliminatoires, accompagnés d'une règle qui dit à l'avance quelle architecture est retenue selon les résultats obtenus. C'est cette antériorité qui fait la valeur de la démarche, car le jour où les campagnes seront menées, la réponse ne se négociera pas : elle se lira dans les tableaux. Si l'architecture proposée satisfait ses critères, elle est retenue ; si elle échoue sur le gain ou sur le délai, c'est le pool local qui est recommandé, avec la fragilité que l'on sait. Mener ces campagnes suppose d'abord d'implémenter les deux organisations retenues, qui sont décrites mais absentes du code. Au-delà, trois prolongements se dessinent : remplacer les proportions de copies fixées a priori par celles de copies réelles anonymisées, commander le nombre de correcteurs actifs par la profondeur de la file si l'architecture proposée est retenue, et mesurer sur un second poste physique l'apport réel d'un nœud supplémentaire, que le dispositif à nœuds émulés ne peut pas établir.

Références bibliographiques

- [1] S. Wasik, M. Antezak, J. Badura, A. Laskowski et T. Sternal, « A Survey on Online Judge Systems and Their Applications », *ACM Computing Surveys*, vol. 51, n° 1, 2018. DOI : 10.1145/3143560
- [2] P. Livaja Mušac, J. Nakić et A. Sović Kržić, « Automatic Assessment Tools for Grading Coding Assignments : A Systematic Literature Review », *Applied Sciences*, vol. 16, n° 11, p. 5658, 2026. DOI : 10.3390/app16115658
- [3] A. G. Ibrahim, R. P. Lopes, J. Rufino et P. Leitão, « On the Impact of Message Brokers Implementations in the Choreography of Microservices », dans *Optimization, Learning Algorithms and Applications (OL2A 2025)*, Communications in Computer and Information Science, vol. 2617, Springer, 2025. DOI : 10.1007/978-3-032-00137-5_1
- [4] B. Çiftçi et B. Çiloğlugil, « A Review of Comparative Studies on Performance Evaluation of Communication Mechanisms for Microservices », dans *Computational Science and Its Applications (ICCSA 2025)*, Lecture Notes in Computer Science, vol. 15650, Springer, 2025. DOI : 10.1007/978-3-031-96962-1_7
- [5] J. Ji, Y. Fu, R. Jin et Q. Lin, « Designing for Scalability : Building a Universal Serverless Messaging Architecture with Apache RocketMQ », dans *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE 2025)*, Industry Papers, ACM, 2025. DOI : 10.1145/3696630.3728536
- [6] N. Pantelic, L. Matic, L. Jakovljevic, S. Eric, M. Eric, M. Stefanović et A. Djordjevic, « Benchmarking SQL and NoSQL Persistence in Microservices Under Variable Workloads », *Future Internet*, vol. 18, n° 1, p. 53, 2026. DOI : 10.3390/fi18010053
- [7] H. Z. Došilović et I. Mekterović, « Robust and Scalable Online Code Execution System », dans *43rd International Convention on Information, Communication and Electronic Technology (MIPRO 2020)*, IEEE, p. 1627-1632, 2020. DOI : 10.23919/MIPRO48935.2020.9245310
- [8] C. Drung, J. Wang et N. Guo, « Enhance Performance of Program Automatic Online Judging Systems Using Affinity Algorithm and Queuing Theory in SMP Environment », dans *Proceedings of the 2011 International Conference on Electronic & Mechanical Engineering and Information Technology (EMEIT)*, IEEE, 2011. DOI : 10.1109/EMEIT.2011.6024016
- [9] N. J. Gunther, *Guerrilla Capacity Planning : A Tactical Approach to Planning for Highly Scalable Applications and Services*, Springer, 2007. ISBN 978-3-540-26138-4. Loi de passage à l'échelle universelle, chapitre 4.

Annexes

Annexe A : structure du code de correction actuel

La fonction principale de correction dans CodEval est `process_run`, dans le fichier `backend/app/grading/engine.py`. Son fonctionnement est le suivant :

```
process_run(db, run, sandbox):
    recuperer les exercices de l'evaluation
    recuperer les participations

    pour chaque participation:
        recuperer les soumissions de l'etudiant
        pour chaque exercice:
            si deja corrige: passer
            code = soumission de l'etudiant ou chaine vide
            resultat = grade_exercice(code, exercice, tests, sandbox)
            enregistrer le resultat en base
            incrementer le compteur de copies traitees
        valider la transaction
```

La fonction `grade_exercice` aiguille selon le type d'exercice :

- QCM, vrai/faux, correspondance et réponse courte : comparaison directe des réponses ;
- code C : compilation avec gcc, exécution en sandbox contre les jeux de tests ;
- algorithmique : transpilation du pseudo-code en Python, exécution en sandbox.

Annexe B : contenu de l'évaluation de mesure

L'évaluation BENCH-SRIT, décrite en section 4.2.1, telle qu'elle est chargée en base par le banc.

Ex.	Titre	Type	Points	Tests officiels	Barème complémentaire
1	Calculatrice en boucle	Code C	6	6 (1 point chacun)	-
2	Premier pointeur sur un entier	Code C	4	1 (3 points)	1 critère de déclaration (1 point)
3	Moyenne de N notes	Algorithmique	4	2 (2 points chacun)	-
4	Notions de base du C	QCM	3	-	5 questions à choix unique
5	Vrai ou faux sur les pointeurs	Vrai/faux	3	-	5 affirmations

Tableau B.1 - Contenu de l'évaluation BENCH-SRIT

Total : 20 points, 5 réponses par copie, dont 3 demandent une exécution en sandbox.

Limites d'exécution, celles de CodEval par défaut : 2 000 ms par test, 5 secondes de temps processeur et 20 secondes pour la compilation. Le matériel et les versions de logiciels ne sont pas repris ici : ils figurent en section 4.1, et le banc les enregistre à chaque mesure.

Annexe C : reproduire le jeu de données

Les scripts sont livrés avec le code, dans `docs/recherche/bench/` :

Fichier	Rôle
dataset_spec.py	Contenu de BENCH-SRIT et variantes de réponses des classes K1 à K5
gen_dataset.py	Tire les copies à partir d'une graine, écrit le fichier et son empreinte
load_dataset.py	Charge le jeu de copies dans une base CodeVal

Tableau C.1 - Scripts de construction du jeu de données

Les trois commandes. La première crée la base modèle et son schéma. Le schéma ne vient pas d'alembic `upgrade head` : la migration de CodeVal modifie des tables existantes, elle ne les crée pas. Il faut `init_db()`, puis `alembic stamp head` pour noter que la base est à jour des migrations.

```
createdb -O codeval codeval_bench_tpl

cd backend
export
CODEVAL_DATABASE_URL=postgresql+psycopg://codeval:codeval@localhost:5432/codeval_bench_tpl
.venv/bin/python -c "from app.db import init_db; init_db()"
.venv/bin/alembic stamp head
```

La deuxième tire les copies, la troisième les charge.

```
cd docs/recherche/bench

python3 gen_dataset.py --seed 2026 --copies 500

CODEVAL_DATABASE_URL=postgresql+psycopg://codeval:codeval@localhost:5432/codeval_bench_tpl \
    ../../../../backend/.venv/bin/python load_dataset.py --dataset data/dataset_v1.json
```

Le tirage affiche la répartition obtenue et l'empreinte SHA-256 du fichier. Le chargement crée un établissement, une enseignante, une classe, les comptes étudiants, l'évaluation avec ses exercices et ses tests, puis une participation par étudiant et une soumission par exercice.

Ce qui se règle.

Réglage	Où
Nombre de copies	--copies (500 par défaut)
Graine du tirage	--seed (2026 par défaut)
Fichier produit	--out
Proportions des classes K1 à K5	CLASS_WEIGHTS, en tête de dataset_spec.py
Exercices, tests et barèmes	EXERCISES, dans dataset_spec.py
Variantes de réponses	dataset_spec.py, une section par exercice

Tableau C.2 - Réglages du tirage des copies

À graine égale, le tirage donne le même fichier sur n'importe quelle machine. Toute modification, ne serait-ce que d'une proportion, change l'empreinte du fichier : les mesures faites avant et après ne sont plus comparables, et l'empreinte enregistrée avec chaque mesure rend cette rupture visible. Une série de mesures se fait donc avec un jeu unique, gelé du début à la fin.

Annexe D : formules utilisées

Les indicateurs sont définis en section 4.4.2. Cette annexe donne leur calcul exact. t_0 est l'heure de création de la campagne, et chaque note porte son heure d'écriture dans la colonne `written_at`.

Temps total (M1).

$$T_{\text{total}} = \text{derniere heure d'écriture d'une note} - t_0$$

Débit (M2).

$$\text{debit} = \text{nombre de copies corrigées} / T_{\text{total}} \times 60$$

Latence par copie (M3). Pour chaque copie, la latence est l'écart entre t_0 et l'heure d'écriture de sa dernière note. Les p_{50} et p_{95} sont les percentiles 50 et 95 de ces latences.

Accélération (M4). Elle se mesure toujours contre la correction séquentielle, à charge égale, et non contre l'architecture testée avec un seul worker.

$$S(N) = T_{\text{total}} \text{ du séquentiel} / T_{\text{total}} \text{ de l'architecture avec } N \text{ workers}$$

Efficacité (M5).

$$E(N) = S(N) / N$$

Une efficacité de 1 correspond à un gain parfait, chaque worker ajouté apportant tout son travail. En dessous, la différence vient de la part sérialisée du traitement, de la coordination entre workers et de la base partagée.

Occupation du processeur (M6). L'échantillonneur additionne les pourcentages des workers et de leurs fils, où 100 % représente un cœur saturé. Pour obtenir l'occupation de la machine, il faut diviser par le nombre de cœurs : sur le poste à 8 cœurs, un relevé de 214 % vaut 2,14 cœurs, soit 27 % de la machine. Les seuils énoncés en pourcentage, comme celui de H1, s'entendent en pourcentage de la machine.

Exactitude (M9). Part des couples (copie, exercice) dont la note et le statut sont identiques à ceux de la référence séquentielle, la comparaison se faisant dans les deux sens pour qu'une note en trop apparaisse comme une note manquante.

Loi de passage à l'échelle universelle. Proposée par Gunther [9], elle décrit le débit obtenu avec N workers.

$$X(N) = \lambda \times N / (1 + \sigma \times (N - 1) + \kappa \times N \times (N - 1))$$

λ est le débit d'un seul worker, σ la part du travail qui reste sérialisée, κ le coût de la coordination entre workers, qui croît comme le nombre de paires. Les trois paramètres sont estimés par moindres carrés. Le nombre de workers au-delà duquel en ajouter fait baisser le débit vaut

$$N_{\text{pic}} = \text{racine}((1 - \sigma) / \kappa)$$

La loi d'Amdahl est le cas particulier de cette loi lorsque κ vaut zéro, c'est-à-dire lorsque la coordination est supposée gratuite. Nous retenons la forme complète, car la coordination entre workers et les accès à la base partagée sont précisément ce que nous cherchons à mesurer.

Régression de H1. Le temps total est ajusté par une droite sur le nombre de copies. La pente donne le coût marginal d'une copie, en secondes, et le coefficient de détermination indique si la croissance est bien linéaire.

```
R2 = 1 - somme des carres des residus / somme des carres des ecarts a la moyenne
```

Traitement des répétitions. Pour chaque configuration, nous retenons la médiane des cinq répétitions analysées, et son intervalle de confiance à 95 % obtenu par rééchantillonnage, avec 4 000 tirages avec remise.

Comparaison de deux architectures. Le test de Mann-Whitney bilatéral, au seuil de 0,05, ne suppose aucune loi normale. Avec cinq répétitions par groupe, la plus petite valeur p atteignable est 0,008. La taille de l'effet est donnée par le delta de Cliff, dont la valeur absolue est dite grande au-delà de 0,474.

Ces calculs sont implémentés dans `docs/recherche/bench/stats.py`, en Python pur.

Annexe E : scripts du banc et lancement des mesures

Fichier	Rôle
<code>dataset_spec.py</code>	Contenu de BENCH-SRIT et variantes de réponses des classes K1 à K5
<code>gen_dataset.py</code>	Tire les copies à partir d'une graine, écrit le fichier et son empreinte
<code>load_dataset.py</code>	Charge le jeu de copies dans une base CodEval
<code>orchestrate.py</code>	Enchaîne les huit étapes d'une mesure pour chaque configuration
<code>sampler.py</code>	Relève le processeur, la mémoire et les attentes de verrous (M6, M7, M8)
<code>stats.py</code>	Percentiles, intervalle bootstrap, ajustement USL, Mann-Whitney, delta de Cliff
<code>run_all.py</code>	Série préliminaire sur la granularité et décomposition par étape (section 5.3)
<code>core.py</code>	Enveloppe appelant le moteur de correction et la sandbox réels
<code>dbworker.py</code>	Processus correcteur de la série préliminaire, réservation SKIP LOCKED

Tableau E.1 - Scripts du banc de mesure

Avant la première mesure. La base modèle doit porter l'horodatage des notes, qui n'existe pas dans le schéma de CodEval et sans lequel aucune durée n'est calculable. `clock_timestamp()` donne l'heure exacte de l'écriture, et non l'heure d'ouverture de la transaction.

```
psql -d codeval_bench_tpl -c "
ALTER TABLE correction_results
ADD COLUMN written_at timestamptz NOT NULL DEFAULT clock_timestamp();"
```

Lancer une série.

```
cd docs/recherche/bench
../../../../backend/.venv/bin/python orchestrate.py --arch seq --charges 10,50,100,200 --
repetitions 6
```

Ce qui se règle.

Réglage	Option
Architecture mesurée	<code>--arch</code>

Réglage	Option
Nombre de workers, valeurs successives	--workers (1 par défaut)
Charges, en copies	--charges (10,50,100,200,500)
Répétitions, la première servant de chauffe	--repetitions (6 par défaut)
Graine de l'ordre de passage	--graine (2026 par défaut)
Abandon d'une mesure	--limite (3 600 s par défaut)
Refroidissement entre deux mesures	--refroidissement (60 s par défaut)

Tableau E.2 - Options de l'orchestrateur

Ce qui est produit. Un fichier par mesure, nommé d'après sa configuration, contenant la configuration, le système, le commit, l'empreinte du jeu, t0, les indicateurs, les notes horodatées, les échantillons et le journal des workers. Puis un fichier d'agrégats et le journal de la campagne.

Système d'exploitation. Les commandes de ces annexes sont identiques sous macOS et sous Linux. Sous Windows, `.venv/bin/python` devient `.venv\Scripts\python.exe` et `VARIABLE=valeur` commande devient `set VARIABLE=valeur` puis la commande.

Table des matières

Pour actualiser la pagination dans Word : sélectionner la table, puis appuyer sur F9 (ou clic droit, « Mettre à jour les champs »).

Sommaire.....	1
Liste des tableaux	3
Liste des figures.....	4
Glossaire.....	5
Sigles et acronymes	7
Introduction générale.....	8
Chapitre 1 : Contexte et problématique.....	9
1.1 Évaluation automatique et CodEval	9
1.2 Problématique de la charge de travail	9
1.3 Question de recherche et hypothèses.....	10
Chapitre 2 : Systèmes étudiés.....	11
2.1 Traitement séquentiel (architecture actuelle)	11
2.1.1 Description	11
2.1.2 Schéma d'architecture.....	12
2.1.3 Avantages	12
2.1.4 Limites.....	12
2.2 Parallélisme sur une machine : pool de processus local (architecture A)	13
2.2.1 Description	13
2.2.2 Schéma d'architecture.....	14
2.2.3 Avantages	14
2.2.4 Limites.....	14
2.3 Workers distribués autour d'un courtier de messages (architecture B)	15
2.3.1 Description	15
2.3.2 Schéma d'architecture.....	16
2.3.3 Avantages	16
2.3.4 Limites.....	16
2.4 Messagerie serverless élastique (architecture C).....	17
2.4.1 Description	17
2.4.2 Schéma d'architecture.....	18
2.4.3 Avantages	18
2.4.4 Limites.....	18

2.5 Discussion sur les limites	19
Chapitre 3 : Architecture proposée.....	21
3.1 Bilan des architectures étudiées	21
3.2 Des limites aux exigences de conception.....	21
3.3 Principe de l'architecture proposée.....	22
3.3.1 Ce que fait CodEval aujourd'hui	22
3.3.2 L'idée de l'architecture proposée	22
3.3.3 Ce qui est ajouté par rapport à l'architecture actuelle.....	23
3.4 Description de l'architecture.....	23
3.4.1 Vue d'ensemble	23
3.4.2 Composants	24
3.4.3 Unité de travail et granularité.....	24
3.4.4 Cycle de vie d'une tâche	25
3.4.5 Déroulement d'une campagne	25
3.4.6 Déploiement	26
3.5 Comparaison avec les architectures étudiées	26
Chapitre 4 : Méthodologie expérimentale	28
4.1 Matériel et logiciels.....	28
4.1.1 Poste de mesure	28
4.1.2 Logiciels	28
4.1.3 Instruments de mesure.....	29
4.1.4 Conditions de mesure	29
4.2 Données corrigées pendant les mesures	30
4.2.1 L'évaluation de référence	30
4.2.2 Le contenu des copies.....	30
4.2.3 Les profils de charge	31
4.3 Construction du jeu de données.....	31
4.4 Ce que nous mesurons.....	33
4.4.1 Les questions posées	33
4.4.2 Les indicateurs.....	34
4.4.3 Les seuils de réussite	34
4.4.4 Plan d'ensemble des mesures.....	35
4.5 Mesure de l'architecture actuelle (référence)	36
4.5.1 Préparation, une seule fois.....	36

4.5.2 Déroulement d'une mesure	36
4.5.3 Mesure avec panne (X4).....	40
4.5.4 Vérification des notes (X5)	40
4.5.5 Réactivité de l'API (X6)	40
4.5.6 Traitement des mesures	41
4.5.7 Traçabilité.....	41
4.6 Mesure de l'architecture A (pool de processus local).....	41
4.7 Mesure de l'architecture B (workers distribués via Redis).....	41
4.8 Mesure de l'architecture proposée (P)	42
4.9 Pourquoi l'architecture C n'est pas mesurée	43
4.10 Limites du dispositif.....	43
Chapitre 5 : Résultats	44
5.1 État d'avancement.....	44
5.2 Vérification du dispositif.....	44
5.3 Mesures déjà disponibles : effet de la granularité	45
5.4 Temps total par charge (X1).....	46
5.5 Gain par worker ajouté (X2)	46
5.6 Passage à deux nœuds (X3).....	47
5.7 Reprise après panne (X4)	47
5.8 Exactitude des notes (X5).....	47
5.9 Réactivité de l'API (X6)	47
5.10 Décision sur les critères.....	48
5.11 Décision sur les hypothèses.....	48
Chapitre 6 : Discussion.....	49
6.1 Analyse des résultats	49
6.1.1 Le traitement séquentiel est inefficace pour le processeur.....	49
6.1.2 L'accélération n'est pas parfaitement linéaire.....	49
6.1.3 L'architecture distribuée dépasse le pool local à nombre égal de workers.....	49
6.2 Vérification des hypothèses.....	49
6.2.1 Hypothèse H1 : le traitement séquentiel est un goulot d'étranglement	49
6.2.2 Hypothèse H2 : le parallélisme intra-nœud réduit le temps de correction	50
6.2.3 Hypothèse H3 : le parallélisme inter-nœuds permet une scalabilité quasi proportionnelle	50
6.3 Limites et implications	50
6.3.1 Limites de l'étude	50

6.3.2 Implications pratiques	50
6.3.3 Implications pour la conception	51
Conclusion générale	52
Références bibliographiques	53
Annexes	54
Annexe A : structure du code de correction actuel	54
Annexe B : contenu de l'évaluation de mesure	54
Annexe C : reproduire le jeu de données	54
Annexe D : formules utilisées	55
Annexe E : scripts du banc et lancement des mesures	57